IOT SMART ENERGY MANAGEMENT FOR CLASSROOM

AFIQ IKHMAL BIN AHMAD RASHIDI

Report submitted in partial fulfillment of the requirements
for the award of the Bachelor of
Information Technology

Faculty of Creative Multimedia & Computing
SELANGOR ISLAMIC UNIVERSITY (UIS)

2026/2027

Appendix D

FAKULTI MULTIMEDIA KREATIF & KOMPUTERAN (FMKK), UNIVERSITI ISLAM SELANGOR (UIS)

DECLARATION OF THESIS AND COPYRIGHT

Author's Full Name : AFIQ IKHMAL BIN AHMAD RASHIDI

Identification Card No : 040104-10-0257

Title : IOT SMART ENERGY MANAGEMENT FOR CLASSROOM

Academic Session : I 2026/2027

I declare that this thesis is classified as:

☐ **OPEN ACCESS** I agree that my thesis to be published as online open access (Full text)

I acknowledge that FMKK, UIS reserve the right as follows:

1. The thesis and final prototype is the Property of FMKK, UIS.
2. The library of UIS has the right to make copies for the purpose of research only.
3. The library has the right to make copies of the thesis for academic exchange.

Certified by:

(Author's Signature) (Supervisor's Signature)

Afiq Ikhmal bin Ahmad Rashidi Ts. Marziana Binti Abdul Majid

Date: Date:

ACKNOWLEDGEMENTS

Bismillahirrahmanirrahim**.** Alhamdulillah, all praise and gratitude be to Allah SWT, the Most Gracious and Most Merciful, for His blessings, strength, and guidance that enabled me to successfully complete my Final Year Project, titled IoT Smart Classroom System. Although I encountered several technical hurdles—including hardware troubleshooting, Wi-Fi connectivity issues, and time management constraints—His grace allowed me to persevere and bring this project to fruition.

First and foremost, I would like to express my deepest gratitude and sincere appreciation to my esteemed supervisor, Ts. Marziana Binti Abdul Majid. Her continuous guidance, valuable advice, and unwavering support were instrumental throughout the development of this project. Her constructive feedback, patience, and domain expertise in IoT applications, microcontrollers, and web development significantly elevated both the system implementation and the quality of this report. I am truly honored to have had her guidance.

I am also deeply grateful to all the lecturers and faculty members who have contributed to my academic growth throughout my studies, equipping me with the knowledge necessary to undertake this endeavor.

My heartfelt thanks go to my beloved parents for their endless love, prayers, and sacrifices, which served as my ultimate source of strength and motivation. I am equally thankful to my siblings and friends for their continuous moral support and encouragement during challenging moments.

Finally, I extend my appreciation to everyone who directly or indirectly contributed to the completion of this project. May Allah SWT reward all of you with continuous blessings and success in this world and the hereafter.

ABSTRACT

The implementation of the IoT Smart Classroom System introduces a modern and efficient Internet of Things (IoT) solution to address common challenges in educational facility management and energy conservation. Traditional classroom management often relies on manual monitoring, which can lead to lights and fans being left turned on in empty rooms, lack of real-time occupancy data, and inefficient electrical energy usage. These limitations result in unnecessary energy wastage and higher operational costs. To overcome these challenges, this project presents an IoT-based automation and monitoring system that enables real-time tracking of classroom occupancy, motion, and light conditions. The system integrates environmental and presence sensors with a microcontroller to collect and transmit data to a web-based dashboard and MySQL database for remote monitoring and analytics. When human entry or motion is detected alongside low ambient light, the system automatically triggers relay modules to activate lamps and fans, ensuring optimal classroom comfort without constant manual intervention. The development of this system utilizes an Arduino Uno microcontroller, dual Infrared (IR) sensors for directional user counting, a Passive Infrared (PIR) sensor for motion detection, and a Light Dependent Resistor (LDR) sensor to measure light intensity. Sensor data is transmitted via an ESP-01 Wi-Fi module to a local database server, enabling administrators to monitor classroom status, receive idle device alerts, and analyze usage trends via a Highcharts occupancy heatmap on the web interface. Additional features such as manual web controls (AUTO, FORCE ON, FORCE OFF) are integrated to provide flexible control options. Furthermore, the IoT Smart Classroom System enhances operational efficiency by reducing manual workload, optimizing energy usage, and providing real-time data visibility. Overall, this project demonstrates how IoT-based solutions can significantly improve campus facility management processes and contribute to smart educational infrastructure.

ABSTRAK

Pelaksanaan IoT Smart Classroom System memperkenalkan satu penyelesaian moden dan efisien berasaskan Internet of Things (IoT) bagi menangani cabaran dalam pengurusan fasiliti bilik darjah dan penjimatan tenaga elektrik. Pengurusan bilik darjah secara tradisional yang bergantung kepada kawalan manual sering menyebabkan lampu dan kipas dibiarkan terpasang di dalam bilik yang kosong, kekurangan data bilangan pengguna secara masa nyata, serta penggunaan tenaga elektrik yang tidak cekap. Kekangan ini boleh mengakibatkan pembaziran tenaga dan peningkatan kos operasi. Bagi mengatasi masalah tersebut, projek ini membangunkan satu sistem automasi dan pemantauan berasaskan IoT yang membolehkan pemerhatian masa nyata terhadap bilangan pengguna, pergerakan, dan tahap pencahayaan bilik darjah. Sistem ini mengintegrasikan sensor persekitaran dengan mikropengawal untuk mengumpul dan menghantar data ke pangkalan data MySQL serta paparan dashboard web bagi tujuan pemantauan dan analisis jarak jauh. Apabila kehadiran pengguna atau pergerakan dikesan bersama keadaan bilik yang gelap, sistem akan mengaktifkan modul relay secara automatik untuk menghidupkan lampu dan kipas bagi memastikan keselesaan ruang dapat dikekalkan tanpa memerlukan kawalan manual berterusan. Pembangunan sistem ini menggunakan Arduino Uno sebagai mikropengawal utama, dwisensor Inframerah (IR) untuk pengiraan arah pengguna, sensor Passive Infrared (PIR) bagi mengesan pergerakan, serta sensor Light Dependent Resistor (LDR) untuk mengukur keamatan cahaya. Data yang dikumpul dihantar melalui modul Wi-Fi ESP-01 ke pangkalan data, membolehkan pentadbir memantau keadaan bilik darjah, menerima amaran peranti terbiar (idle alert), serta menganalisis trend penggunaan melalui graf heatmap kepadatan kelas pada antaramuka web. Kawalan tambahan seperti butang pintas manual pada web (AUTO, FORCE ON, FORCE OFF) turut diintegrasikan bagi menyokong fleksibiliti kawalan. Secara keseluruhannya, IoT Smart Classroom System membantu meningkatkan kecekapan pengurusan fasiliti dengan mengurangkan beban kerja manual, mengoptimumkan penggunaan tenaga, serta menyediakan data masa nyata untuk pemantauan yang lebih sistematik. Projek ini menyumbang kepada pembangunan kampus pintar dan menyokong amalan kelestarian tenaga melalui penggunaan teknologi digital moden.

TABLE OF CONTENTS

	Page
TITLE PAGE	i
DECLARATION	ii
ACKNOWLEDGEMENTS	iii
ABSTRACT	iv
ABSTRAK	v
TABLE OF CONTENTS	vi
LIST OF TABLES	x
LIST OF FIGURES	xi

CHAPTER 1 INTRODUCTION

1.1	Introduction of the Chapter	1
1.2	Project Introduction	1
1.3	Problem Statement	2
1.3	Project Objective	3
1.4	Project Scope	3
	1.4.1 Target Users	3
	1.4.2 Project Limitation	4
1.5	Project Contribution	5
1.6	Chapter Conclusion	5

CHAPTER 2 LITERATURE REVIEW

2.1 Introduction 6

2.2 Domain Studies 6

2.2.1 Internet of Things (IoT) 6

2.2.2 Web-Based Systems 7

2.2.3 Classroom Automation 8

2.3 Technology Studies 10

2.3.1 Arduino Uno 10

2.3.2 ESP-01 Wi-Fi Module 10

2.3.3 Sensors Array 11

2.3.4 Relay Module and Equipment 11

2.3.5 Web Dashboard & Database 12

2.4 Software Specification 13

2.5 Hardware Specification 14

2.6 Existing Projects 15

2.7 Chapter Conclusion 20

CHAPTER 3 METHODOLOGY

3.1 Introduction 21

3.2 Project Methodology 22

3.3 Conceptual Framework 23

3.4 Project Design 25

3.4.1 Network Diagram 25

3.4.2 Use Case Diagram 26

3.4.3 ER Diagram 27

3.4.4 Flowchart 28
3.5 Gantt Chart 30
3.6 Conclusion 31

CHAPTER 4 RESULTS AND DISCUSSION

4.1 Introduction 32
4.2 Project Impelementation 32
4.3 Results 42
4.3.1 Performance Evaluation of Hardware and IoT System 42
4.3.2 Web Dashboard and Analytics 43
4.3.3 Comparison with Project Objectives 44
4.3.4 User Testing and System Usability Feedback 45
4.4 Conclusion 45
4.4.1 Success Analysis 45
4.4.2 Challenges Faced 46
4.4.3 System Limitations 46
4.4.4 Future Improvements 47
4.5 Conclusion of Chapter 47

CHAPTER 5 CONCLUSION AND RECOMMENDATIONS

5.1 Introduction 48
5.2 Lesson Learned 48
5.3 Recommendations for Future Project Implementation 49

5.4 Conclusion 49

5.5 Conclusion of Chapter 50

REFERENCES 51

APPENDICES 54

Action Script/Programming 54

LIST OF TABLES

Table No.	Title	Page
2.1	Table of Hardware Specification	14
2.2	Table of Comparison Existing Projects	19
4.1	Table of Connection ESP-01 to Arduino Uno	35
4.2	Table of Connection ESP-01 to Arduino Uno	36
4.3	Table of Connection Dual IR Sensor to Arduino Uno	36
4.4	Table of Connection LDR Sensor to Arduino Uno	36
4.5	Table of Connection PIR Sensor to Arduino Uno	37
4.6	Table of Connection Relay Module to Arduino Uno	37
4.7	Table of Connection LCD I2C 16x2 to Arduino Uno	37

LIST OF FIGURES

Table No.	Title	Page
2.1	Energy Smart Saving Classroom – UTM	15
2.2.1	Smart Classroom Electricity Saving – Unimap	16
2.2.2	Smart Classroom Electricity Saving – Unimap	16
2.3	NodeMCU+ Blynk Protoytpe – Indonesia	17
2.4	Smart Classroom System – IJERT	18
3.1	Network Design	25
3.2	Use Case Diagram	26
3.3	ER Diagram	27
3.4	Flowchart IoT System	28
3.5	Flowchart Admin Login	29
3.6	Gantt Chart	30
4.1	IR, PIR, LDR Sensors	38
4.2	Relay Module	38

4.3	LCD I2C 16x2	38
4.4	ESP-01	39
4.5	Fan & Light	39
4.6	Breadboard Connector for All Components	39
4.7	Arduino Uno	40
4.8	Finish Setup	40
4.9	View From Front Classroom	41
4.10	View from Back Classroom	41
4.11	Web Dashboard	43
4.12	Heatmap Chart	44

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION OF THE CHAPTER

This chapter explains the overall overview of the IoT Smart Energy Management System for Classroom project. It covers the background of the study, the importance of using Internet of Things (IoT) technology in education, and the basis for developing an automated smart classroom system.

With the advancement of digital technology, the education sector has also undergone changes towards the use of smart systems to enhance operational efficiency. Therefore, the integration of IoT technology allows classrooms to become more automated, energy-efficient, responsive, and easy to monitor.

This chapter discusses the project introduction, problem statement, project objectives, project scope, contributions, and chapter conclusion as preparation for the subsequent chapters.

1.2 PROJECT INTRODUCTION

The IoT Smart Energy Management System for Classroom is developed to optimize classroom facility management and reduce electricity wastage through the integration of Internet of Things (IoT) technology. This system seamlessly connects hardware components, including dual Infrared (IR) sensors, a Passive Infrared (PIR) motion sensor, a Light Dependent Resistor (LDR) sensor, an Arduino Uno

microcontroller, an ESP-01 Wi-Fi module, a 16x2 I2C LCD, and a 2-channel Relay Module with a local PHP web application and MySQL database.

In most educational institutions, classroom appliances such as lights and fans are controlled manually. This often leads to severe energy wastage when appliances are left running continuously in empty classrooms. To solve this, the proposed system utilizes dual IR sensors placed at the classroom doorway to perform sequential directional counting. Based on entry and exit sequences, Arduino Uno dynamically updates the real-time occupancy count, which is displayed locally on a 16x2 I2C LCD display.

The system executes automated appliance control logic via an active LOW Relay Module. In AUTO Mode, the fan is automatically activated when room occupancy is greater than zero or when motion is detected by the PIR sensor. Concurrently, the lights are activated only when occupancy/motion is present and the ambient lighting is detected as dark by the LDR sensor. To handle temporary inactivity and empty room states, two timing windows are enforced: a 10-second device timeout that switches off appliances if no motion is detected, and a 15-second reset timeout that automatically resets the occupancy count to zero if the room remains completely vacant.

Furthermore, the system features a manual Web Override mechanism. Administrators can toggle between AUTO (0), FORCE ON (1), and FORCE OFF (2) via the web dashboard. The ESP8266 module polls control commands from backhend php file every 1.5 seconds and forwards them to Arduino. Sensor data and appliance status such as total count, ldr, pir, relay light and relay fan are transmitted wirelessly via HTTP POST to backhend php file both immediately upon state changes and periodically every 60 seconds. The centralized web dashboard displays real-time status cards, auto-refreshes every 2 seconds, provides an automated visual warning banner when appliances are left ON in an empty room for ≥ 5 seconds, and renders historical occupancy patterns using a Highcharts Heatmap chart.

1.3 PROBLEM STATEMENT

1.3.1 Classroom lights and fans rely entirely on manual physical operation due to the absence of an automated real-time occupancy monitoring system.

1.3.2 Electrical appliances are frequently left running in empty classrooms, causing unnecessary power consumption and higher operational costs.
1.3.3 There is a lack of a centralized web dashboard and database system to track occupancy trends, monitor device statuses, and analyze usage patterns.

1.4 PROJECT OBJECTIVES

1.4.1 To design an IoT architecture integrating dual IR, PIR, LDR sensors, Arduino Uno, ESP8266 and relay modules for classroom occupancy detection.

1.4.2 To develop an automated hardware prototype that dynamically controls lights and fans based on real-time user counts and ambient light levels.

1.4.3 To implement a web dashboard and MySQL database for real-time monitoring, manual override controls and historical occupancy analytics.

1.5 PROJECT SCOPE

1.5.1 Target Users

1.5.1.1 Classroom Users

Students, academic staff, and visitors who occupy the classroom. Their presence triggers sensor inputs to automate lighting and ventilation without requiring manual interaction with physical switches.

1.5.1.2 System Administrators

Facility management personnel who access the web dashboard to monitor real-time user counts, appliance operating states, ambient light status, motion states, idle-device warning alerts, weekly heatmap trends, and toggle manual control modes.

1.5.2 Project Limitations

- The system is developed as a single-classroom prototype using basic microcontrollers (Arduino Uno and ESP-01) and standard sensors, excluding industrial-grade automation equipment due to student project time and budget constraints.
- System functionality is limited to basic appliance automation (lights and fans) via relay modules triggered by dual IR directional counting, PIR motion, and LDR ambient light sensing, without incorporating advanced artificial intelligence or predictive analytics.
- Project implementation and testing are restricted to a localized prototype for a single main entrance, limiting large-scale deployment and extensive field testing across multiple facilities.

1.6 PROJECT CONTRIBUTIONS

This project contributes an end-to-end IoT smart classroom solution designed to eliminate electricity wastage and modernize facility management. It establishes a dual-microcontroller architecture where the Arduino Uno handles low-level sensor polling (IR, PIR, LDR), LCD rendering, and relay switching, while the ESP8266 manages Wi-Fi connectivity, HTTP POST data transmission to backhend file php to store the data, and HTTP GET polling from backhend file php to get latest control mode value. The primary contribution is the multi-layered automation logic, integrating directional IR counting, PIR motion timeout protection within 10 seconds to 15 seconds, and LDR light sensitivity to ensure lighting operates only when necessary. Additionally, the centralized web dashboard introduces real-time appliance monitoring, an automated 5-second idle-device alert banner, manual web override modes (AUTO/ON/OFF), and a Highcharts Heatmap for weekly classroom occupancy analytics, serving as a cost-effective blueprint for smart campus energy management.

1.7 CHAPTER CONCLUSION

This chapter discussed the background, problem statement, objectives, scope, and contributions of the IoT Smart Classroom project. The project aims to transform traditional classrooms into smart classrooms through automation, real-time monitoring, data storage, and IoT-based control. The next chapter will discuss the Literature Review, covering technologies and related studies that support the development of this project.

CHAPTER 2

LITERATURE REVIEW

2.1 INTRODUCTION OF THE CHAPTER

This chapter establishes the foundational research for the IoT Smart Classroom project through a structured approach. It begins by discussing the theoretical background of IoT and classroom automation, followed by an evaluation of the required hardware and software components. The chapter then reviews related projects and conducts a comparative analysis to identify limitations and potential improvements. Overall, this chapter progresses from conceptual understanding to practical implementation considerations, providing a comprehensive framework to support the development of the proposed system.

2.2 DOMAIN STUDY

2.2.1 Internet of Things (IoT)

The Internet of Things (IoT) refers to a network of interconnected physical devices embedded with sensors, microcontrollers, software, and network connectivity that collect, exchange, and process data through the Internet to enable intelligent monitoring, control, and automation in various domains including education, healthcare, and industrial systems. IoT systems typically comprise sensors, microcontrollers, communication modules, and cloud or web services that facilitate remote access and real-time decision-making. In smart environments, IoT plays a vital role by automating routine

tasks and reducing human intervention while improving operational efficiency. Research shows that IoT technology enables automatic data collection from sensor networks and communication with remote servers for processing and action (Al-Mutawa & Eassa, 2020).

In the context of smart classrooms, IoT enables the automatic detection of environmental conditions and user presence, which can be used to control devices such as lighting and fans based on predefined logic. For example, when occupancy or motion is detected through sensors, equipment can automatically be turned on, and conversely turned off automatically when the room becomes completely vacant to conserve energy. Studies on classroom automation systems have demonstrated that IoT-based solutions using motion and environmental sensors can effectively optimize energy use and enhance the learning environment (Ahmed et al., 2024).

In this project, the IoT implementation extends beyond basic one-way data collection by establishing continuous two-way interaction between the local physical environment and the web server. Sensor data including directional IR occupancy counts, PIR motion states, and ambient LDR light levels are captured in real time by an Arduino Uno and transmitted wirelessly by an ESP8266 module to a centralized PHP/MySQL database via HTTP POST requests. Concurrently, the system fetches remote manual override commands from the web server via HTTP GET requests every 1.5 seconds, demonstrating a responsive and robust IoT architecture for smart classroom energy management.

2.2.2 Web-Based Systems

Web-based systems are integral to IoT architectures because they provide centralized interfaces through which users and administrators can monitor, control, and manage connected devices remotely. These systems typically consist of a server hosting a web application, a database for storing data, and a user interface accessible via standard internet browsers. By using web dashboards, administrators can visualize real-time data, configure system settings, and analyze historical records from any location with internet access.

In smart classroom environments, web dashboards enable central monitoring of sensor data, device status, and automation logs. This centralized approach allows for efficient oversight of the system without requiring dedicated software installations on client devices, as the dashboard can be accessed with any browser. Web-based control systems also support scalability, as new features or interfaces can be added without altering the underlying hardware infrastructure.

Additionally, web systems facilitate structured data storage in centralized databases, which supports long-term analytics such as usage patterns and resource consumption trends. Centralized data storage enables institutions to identify optimization opportunities and report on system performance. Therefore, integrating IoT devices with web-based dashboards enhances the usability, flexibility, and analytical capabilities of smart classroom systems.

In this project, the web-based system is implemented using a local PHP and MySQL stack (Laragon) integrated with interactive frontend components. The dashboard (index.php) features an automated 2-second meta-refresh cycle to provide real-time updates of room occupancy, LDR light levels, PIR motion detection, and lamp/fan operational states. To prevent undetected energy waste, the web backend incorporates an automated alert banner that triggers if appliances remain active in an empty room for 5 seconds or longer. Furthermore, the web application provides remote manual override buttons (AUTO, FORCE ON, FORCE OFF), while embedding a Highcharts Heatmap visual chart to display weekly classroom occupancy density profiles across hourly intervals

2.2.3 Classroom Automation

Classroom automation refers to the application of smart technologies to automatically manage physical activities and environmental controls within a classroom. By integrating sensors and control mechanisms, automation aims to reduce manual intervention, improve energy efficiency, and enhance the learning environment. Common sensors used in classroom automation include Passive Infrared (PIR) and Infrared (IR)

detectors that sense motion and presence, triggering automated responses such as turning lights and fans on or off (Suhaimi & Roslan, 2020).

Smart classroom automation systems typically monitor environmental data and occupancy in real time and then execute control logic based on this information. These systems reduce energy consumption by ensuring that devices are active only when needed. For example, if no presence is detected for a defined period, the automation logic can turn off lighting and HVAC equipment, thereby reducing wastage. The integration of automation with remote monitoring platforms enhances transparency and enables administrators to review system performance, plan energy usage strategies, and perform diagnostics more effectively.

Research also shows that IoT-based classroom automation can support advanced features beyond basic control, such as alert notifications and environmental condition tracking, which further contribute to creating adaptive and responsive classroom environments. Overall, classroom automation transforms traditional classrooms into intelligent spaces that respond to user behavior and environmental changes.

In this project, classroom automation is elevated through a multi-sensor fusion control logic executed by an Arduino Uno microcontroller driving a 2-channel active-LOW relay module. The system uses dual IR sensors at the entrance for directional occupancy counting, a PIR motion sensor to verify physical movement inside the room, and an LDR light sensor to detect ambient brightness. Under AUTO mode, the ventilation fan is activated whenever room occupancy is greater than zero or motion is detected, whereas the lights are switched ON only when occupancy/motion is present AND ambient lighting is detected as dark. To handle temporary user inactivity and prevent energy wastage, the automation firmware enforces two non-blocking timing thresholds, a 10-second device timeout that turns off appliances if no motion is detected, and a 15-second reset timeout that automatically resets the room count to zero if the classroom remains vacant, ensuring optimized energy management without manual switch operation.

2.3 TECHNOLOGY STUDY

2.3.1 Arduino Uno

Arduino is an open-source electronics platform based on easy-to-use hardware and software, making it ideal for prototyping microcontroller-based automation systems. In the IoT Smart Energy Management System for Classroom, the Arduino Uno functions as the main local processor and decision-making hub of the system. Powered by the ATmega328P microcontroller, it continuously polls digital input signals from various sensors, evaluates automation logic rules, updates local displays, and controls electrical relay actuators. The board is programmed using the Arduino IDE in C/C++, allowing custom non-blocking control algorithms to be uploaded to the board.

2.3.2 ESP-01 Wi-Fi Module

The ESP8266 Wi-Fi module functions as the dedicated wireless communication bridge between the local Arduino Uno microcontroller and the centralized database server. Operating in Station mode, it establishes a reliable Wi-Fi link and handles continuous two-way HTTP network communication via a 9600-baud SoftwareSerial connection. For outgoing data, the module parses incoming sensor string payloads from the Arduino—containing real-time room occupancy counts, ambient light readings, PIR motion statuses, and relay operational states—and transmits them directly to the web server using HTTP POST requests for continuous logging, alongside a periodic 60-second heartbeat log. For incoming commands, the module periodically polls the server every 1.5 seconds via HTTP GET requests to fetch administrative manual control modes (AUTO, FORCE ON, or FORCE OFF) and forwards these control signals back to the Arduino Uno, while maintaining network resilience through automated reconnection checks.

2.3.3 Sensors Array

The system incorporates a multi-sensor array consisting of dual Infrared (IR) sensors, a Passive Infrared (PIR) motion sensor, and a Light Dependent Resistor (LDR) light sensor to enable precise occupancy tracking and environmental assessment. The dual IR sensors track directional movement at the classroom doorway by evaluating the sequential interruption order between the two beams within a 5-second timing window, dynamically incrementing or decrementing the net occupancy count. Complementing the directional counting, the PIR motion sensor continuously monitors human presence inside the classroom to provide safety fallback logic; if no motion is detected for 10 seconds, active appliances are automatically powered off, and if inactivity persists for 15 seconds, the room count automatically resets to zero. Finally, the digital LDR senso measures ambient room brightness to prevent unnecessary energy consumption by allowing lighting relays to activate only when natural room light falls below the predefined threshold.

2.3.4 Relay Module and Equipment

The 2-channel relay module acts as an electrically operated isolation switch that allows the low-voltage DC signals from the Arduino Uno to safely control higher-voltage AC classroom appliances. Connected to digital output pins 4 (Relay Light) and 5 (Relay Fan), the module operates on active-LOW logic, where a LOW output signal energizes the relay coil to turn the connected load ON, while a HIGH signal de-energizes it to switch the load OFF. Under automated conditions, the relay module regulates the fan based on occupant presence or PIR motion, and controls the lighting circuit by evaluating both presence and ambient LDR dark thresholds. Additionally, it responds directly to administrative override signals, allowing appliances to be forcibly powered ON or OFF regardless of sensor inputs.

2.3.5 Web Dashboard & Database

The web dashboard and backend database serve as the centralized management platform for real-time monitoring, data visualization, and administrative override control. The relational database systematically logs incoming time-series entries—including occupant counts, PIR motion detections, ambient light levels, relay operating states, and timestamps—to maintain complete historical records. Accessible via standard web browsers, the dashboard features live status cards with automated 2-second page refreshes to display real-time classroom conditions. It incorporates an automated visual warning banner that immediately alerts administrators if appliances remain powered ON in an empty room for 5 seconds or longer. Additionally, the platform provides interactive control buttons enabling administrators to switch between automated sensor control and manual override modes, alongside an interactive heatmap chart that aggregates hourly and daily historical logs to analyze weekly classroom occupancy patterns.

2.4 SOFTWARE SPECIFICATIONS

In the IoT Smart Classroom project, several main software components are used to develop an efficient, secure, and centralized system. These software components enable integration between hardware such as Arduino and ESP01 with the database and web dashboard, allowing real-time monitoring and automatic classroom control.

Arduino IDE: Arduino IDE serves as the primary Integrated Development Environment (IDE) used to write, compile, and upload firmware written in C/C++ to the Arduino Uno and ESP8266 microcontrollers. It provides an intuitive interface for managing microcontroller libraries, including SoftwareSerial for establishing UART inter-board communication and LiquidCrystal_I2C for driving the local 16x2 LCD display. Through the Arduino IDE, custom algorithms for non-blocking sensor polling, motion timeout timers, and relay actuation logic are deployed onto the hardware.

Laragon (phpMyAdmin): Laragon provides the local web server environment hosting the Apache HTTP server, PHP execution engine, and MySQL database management system. Included within this stack is phpMyAdmin, a web-based database administration tool used to design, manage, and inspect the relational tables (room, room_log, device_control, and admin). It allows developers to monitor incoming data structures, verify primary and foreign key constraints, and manually test database queries.

Visual Studio Code (VS Code): Visual Studio Code functions as the main source code editor for constructing and debugging the system's web software components. It provides a feature-rich environment for writing backend PHP scripts, configuring HTML structures, applying CSS layouts, and developing JavaScript client-side logic. Its integrated extension support and built-in terminal streamline the workflow when managing server scripts and web assets.

Programming Languages (C++, PHP, JavaScript): The system relies on a combination of programming languages to handle both hardware automation and software interaction. C/C++ is used within the Arduino IDE to execute low-level hardware control, such as reading IR, PIR, and LDR sensor inputs, enforcing timing delays, and toggling relay outputs. PHP handles server-side execution by processing incoming HTTP POST sensor logs from the ESP8266, fetching active manual override modes via HTTP GET requests,

and querying MySQL records. JavaScript operates on the frontend to refresh dashboard elements automatically every 2 seconds and enable dynamic visual responses.

Web Dashboard (HTML, CSS, PHP, Highcharts): The web dashboard acts as the centralized management interface built using HTML5 for markup, CSS3 with SB Admin 2 and Bootstrap frameworks for responsive UI design, and PHP for retrieving live database records. It incorporates real-time status cards, an automated visual alert banner triggered when appliances remain powered ON in empty rooms for 5 seconds or longer, and administrative override buttons (AUTO, ON, OFF). Additionally, it integrates the Highcharts JavaScript library (heatmap.js) to render an interactive 2D Heatmap chart, dynamically displaying hourly and daily classroom occupancy trends to assist in facility management.

2.5 HARDWARE SPECIFICATION

HARDWARE	SPECIFICATION
Laptop 	Lenovo LOQ 15IAX9E Intel Core i5-12450HX 16GB Ram 15.6” (1920x1080) 144Hz NVD RTX3050 6GB Windows 11 MS Office Home

Table 2.1: Table of Hardware Specification

2.6 EXISTING PROJECTS

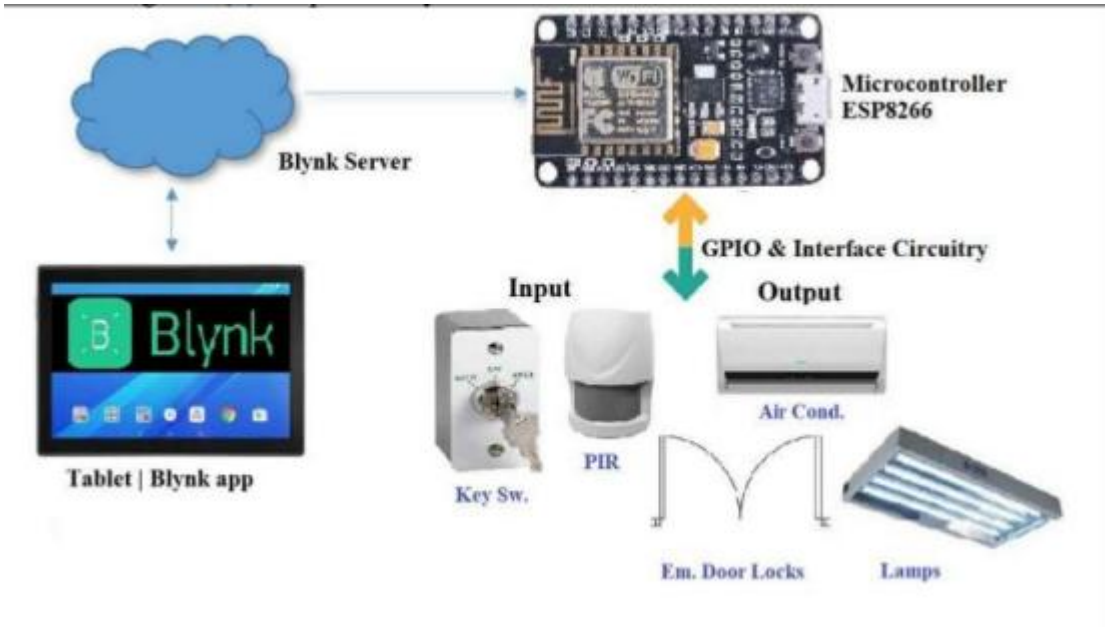


Figure 2.1: Energy Smart Saving Classroom – UTM

The first project is Energy Smart Saving Classroom by UTM, which uses motion detection for light and fan control to improve energy efficiency. The system enables equipment automation and mobile-based monitoring. However, it does not include occupancy counting, centralized database storage, or a web-based dashboard for monitoring (Yusof et al., 2019). The proposed IoT Smart Classroom project improves this by integrating user counting, centralized database logging, web-based dashboard visualization, and basic security mechanisms such as API token authentication.



Figure 2.2.1: Smart Classroom Electricity Saving – UniMAP



Figure 2.2.2: Smart Classroom Electricity Saving – UniMAP

The second project is Smart Classroom for Electricity-Saving with Integrated IoT System at UniMAP, which uses IR sensors to detect user presence and control lights and fans through the Blynk mobile application. The main advantage of this project is equipment automation for energy saving and real-time monitoring via a mobile platform. However, the system does not include centralized web-based monitoring, structured database logging, or enhanced security mechanisms (Rahman et al., 2020).

In comparison, the proposed IoT Smart Classroom project enhances this approach by implementing occupancy-based user counting, centralized database storage, a web-based dashboard for monitoring, and security features such as user authentication and API token validation. These additions provide a more comprehensive and systematically managed smart classroom solution.

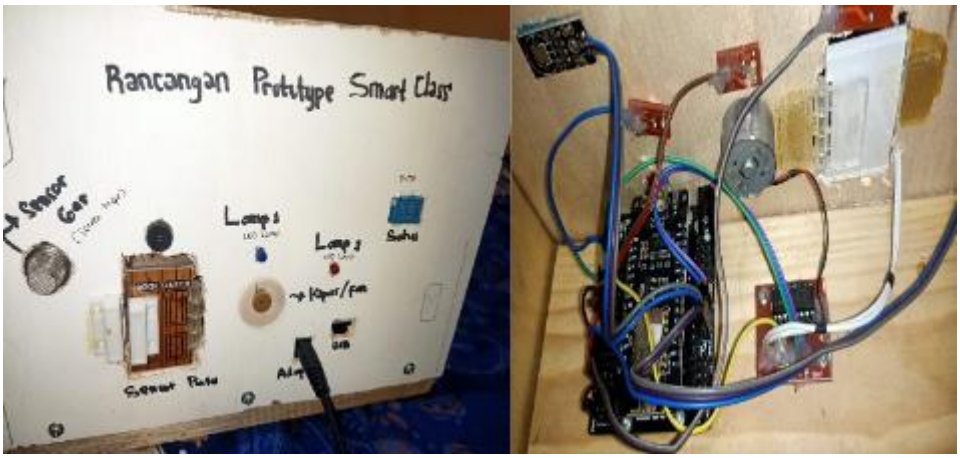


Figure 2.3: NodeMCU + Blynk Prototype – Indonesia

The third project is a NodeMCU + Blynk prototype developed in Indonesia, which focuses on classroom automation using IoT and real-time monitoring through the Blynk application. This system enables real-time equipment control and room monitoring. However, the project does not integrate occupancy-based user counting, does not provide a centralized web dashboard, and lacks sufficient security mechanisms (Pratama & Hidayat, 2021).

In comparison, the proposed IoT Smart Classroom project enhances this approach by implementing IR/PIR-based user counting, centralized database storage, web-based monitoring, and API token authentication, providing a more structured and secure smart classroom system.

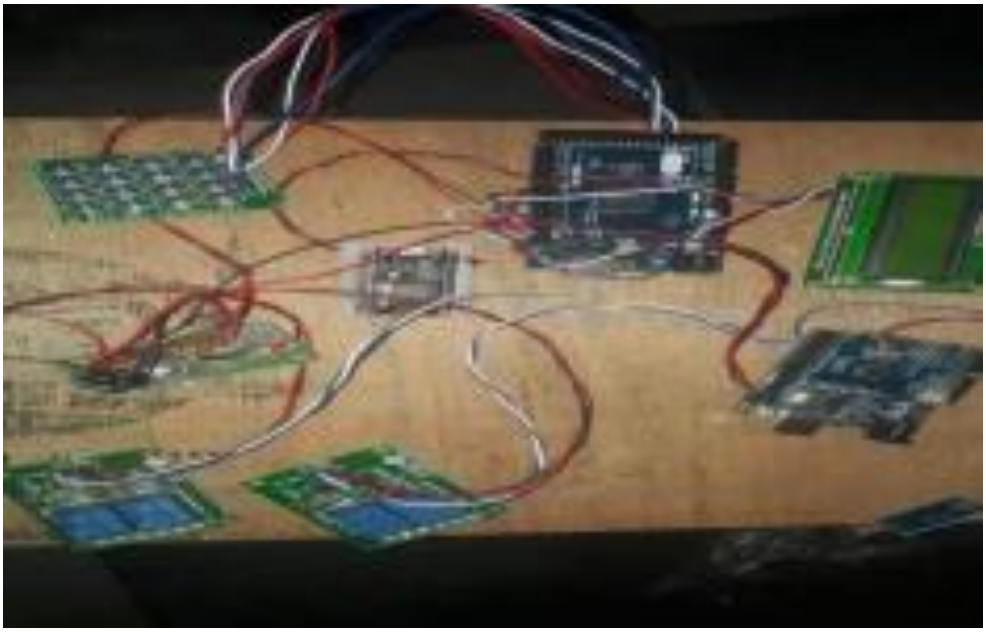


Figure 2.4: Smart Classroom System – IJERT

The fourth project is the Smart Classroom System published in the International Journal of Engineering Research & Technology, which emphasizes manual equipment control through a web-based interface. The main advantage of this system is remote equipment management via web access. However, the project does not integrate occupancy-based user counting and does not store data centrally in a structured database (Patil & Kulkarni, 2018).

In comparison, the proposed IoT Smart Classroom project enhances this approach by implementing occupancy-based user counting, centralized database storage, and security features such as user authentication and API token authentication, providing a more automated and systematically managed smart classroom solution.

Project / Feature	User Counting (Dual IR)	Motion Safety / Validation (PIR)	Ambient Light Sensing (LDR)	Real-Time Monitoring	Web Dashboard	Centralized Database	Web-Based Manual Override Control
Energy Smart Saving – UTM	✗	✓	✗	✓	✗	✗	✗(Blynk App)
Smart Classroom – UniMAP	✗	✗	✗	✓	✗	✗	✗(Blynk App)
NodeMCU + Blynk – Indonesia	✗	✗	✗	✓	✗	✗	✗(Blynk App)
Smart Classroom – IJERT	✗	✗	✗	✓	✓	✗	✓(Manual Web Only)
IoT Smart Classroom (My Project)	✓	✓	✓	✓	✓	✓	✓

Table 2.2: The comparison of existing projects

2.7 CHAPTER CONCLUSION

The literature review shows that the use of IoT improves classroom efficiency through equipment automation, real-time monitoring, and centralized data management. Previous projects provide an important foundation, but most of them do not fully combine occupancy-based user counting, centralized web dashboards, systematic data storage, and security mechanisms within a single architecture.

The proposed IoT Smart Classroom project addresses these gaps by integrating all of these key elements into a cohesive system. Dual IR sensors perform directional user counting through sequential detection at a single entrance, classroom equipment is controlled automatically based on real-time occupancy counts, and activity logs are transmitted wirelessly to a centralized MySQL database. The collected data is presented on a web dashboard for real-time monitoring, while access is protected through an admin user authentication mechanism. This integration produces a more complete, practical, and energy-efficient smart classroom prototype.

This chapter provides the foundation for Chapter 3 – Methodology, which will detail the system design, data flow, and prototype implementation.

CHAPTER 3

PROJECT METHODOLOGY AND TECHNIQUE

3.1 INTRODUCTION

This chapter presents the methodology, architecture, and structural design used in the development of the IoT Smart Energy Management System for Classroom. The purpose of this chapter is to describe the systematic approach, execution models, hardware-software integration processes, and technical diagrams applied to ensure the successful implementation of the proposed system. A well-defined methodology is essential to guide the project development systematically, minimize technical risks, and ensure that all objectives—such as real-time occupancy counting, environmental light detection, motion-based timeout safety, manual web overrides, and visual analytics—are achieved within the specified scope and timeline.

The development of this project involves integrating low-level hardware components (Arduino Uno, dual IR sensors, PIR motion sensor, LDR light sensor, 16x2 I2C LCD display, and a 2-channel relay module) with wireless networking modules (ESP8266 Wi-Fi) and backend web infrastructure (PHP web server, MySQL database, and Highcharts visualization dashboard). Therefore, a structured development process is required to bridge hardware actuation with server-side logging and web application controls.

This chapter explains the selection of the Prototyping Model as the primary system development methodology. It details the multi-layered conceptual framework illustrating how sensor signals transition into local processing, wireless network payloads, database logs, and interactive web dashboard outputs. Furthermore, it provides technical system design diagrams including the Network Diagram, Use Case Diagram, Entity-Relationship Diagram (ERD), and execution flowcharts for both hardware firmware and

web backend processes. Finally, the project timeline and task breakdown are outlined using a Gantt Chart to ensure proper planning and execution across all development phases.

3.2 PROJECT METHODOLOGY

The development of the IoT Smart Energy Management System for Classroom follows the Prototyping Model based on the Software Development Life Cycle (SDLC) framework. This iterative approach is chosen because IoT projects involve continuous hardware-software integration, requiring empirical testing and hardware calibration before achieving a fully functional system. The methodology progresses systematically through six core phases: Requirements Analysis, System Design, Development, Testing, Deployment, and Maintenance.

In the Requirements Analysis phase, system functional and technical requirements are identified. Key specifications include directional entry and exit user counting using dual IR sensors, safety fallback motion detection via a PIR sensor, ambient light evaluation using an LDR sensor, local status display via a 16x2 I2C LCD, active-LOW relay actuator control, automated HTTP POST data logging to MySQL, 1.5-second HTTP GET manual override polling, automated 5-second idle-device alert banners, and interactive Highcharts Heatmap visual analytics.

During the System Design phase, architectural layouts and software structures are established. Hardware schematics map digital pin connections between the Arduino Uno, ESP8266 Wi-Fi module, sensor array, LCD display, and relay module. Simultaneously, software design focuses on structuring relational MySQL database tables, defining SoftwareSerial payload formats, and designing responsive web dashboard wireframes.

In the Development phase, hardware assembly and software coding are executed concurrently. Microcontroller firmware is written in C/C++ using the Arduino IDE to handle non-blocking timers, sensor polling loops, LCD updates, and serial communication. Web backend processing scripts and frontend interfaces are developed using PHP, MySQL, HTML, CSS, and JavaScript within the Laragon server environment.

The Testing phase involves comprehensive functional and integration testing. The prototype is tested to verify directional IR entry and exit counting accuracy, 10-second

PIR device timeouts, 15-second empty room count resets, 1.5-second web override response times, HTTP POST database insertion stability, and 60-second periodic heartbeat logging. Any errors, such as signal bouncing or serial connection drops, are identified and rectified.

During the Deployment phase, the tested prototype and web application are installed into the local classroom environment. Hardware sensors and relays are mounted at the classroom main entrance and appliance load circuits, while the web application is hosted on the local Laragon to enable real-time monitoring and control access for system administrators.

Finally, the Maintenance phase ensures ongoing operational reliability and system stability. This includes regular inspection of physical IR sensor alignment, cleaning of LDR optical surfaces, monitoring Wi-Fi network reconnectivity logic, verifying MySQL database log size, and updating web dashboard analytics over time.

3.3 CONCEPTUAL FRAMEWORK

The conceptual framework of the IoT Smart Energy Management System for Classroom illustrates the structural organization and data flow between physical hardware components, communication modules, backend server scripts, database tables, and the web monitoring dashboard. It defines how raw environmental inputs are transformed into local microcontroller actions, transmitted wirelessly over network protocols, logged into structured database tables, and presented as real-time visual analytics and administrative override controls. The framework is structured into four main operational layers: Input Layer, Processing Layer, Communication & Database Layer, and Output Layer.

In the Input Layer, environmental parameters and human activities are continuously captured by a multi-sensor array. Dual Infrared (IR) sensors installed at the classroom doorway detect physical presence to evaluate directional entry and exit movements. Simultaneously, a Passive Infrared (PIR) motion sensor tracks physical movement within the room to provide motion safety fallback logic, while a Light

Dependent Resistor (LDR) light sensor measures ambient room brightness to determine lighting requirements.

In the Processing Layer, the Arduino Uno microcontroller serves as the primary local control hub. It processes the digital inputs from the dual IR sensors using sequence-matching algorithms to update the net occupancy count and outputs these metrics to a 16x2 I2C LCD display. It also executes non-blocking timing loops using millis() to enforce a 10-second device timeout for powering OFF idling equipment when no motion is detected, as well as a 15-second reset timeout to reset the occupancy count to zero during prolonged room vacancy. Under AUTO mode, the Arduino evaluates sensor conditions to drive a 2-channel active-LOW relay module, powering ON the fan when occupancy or motion is present and powering ON the lights only when presence is detected AND ambient light reads dark.

In the Communication & Database Layer, an ESP8266 Wi-Fi module operates in Station mode to establish a wireless bridge between the local hardware and the server infrastructure. Communicating with the Arduino Uno via a 9600-baud SoftwareSerial connection, the ESP8266 receives formatted sensor string payloads upon state changes or as a 60-second heartbeat log, sending them via HTTP POST requests to simpan.php to insert structured records into the MySQL room_log table. Concurrently, the module sends HTTP GET requests to getLedStatus.php every 1.5 seconds to poll the active control_mode from the device_control table and forward override commands (CTRL:0, CTRL:1, CTRL:2) back to the Arduino Uno.

Finally, in the Output Layer, the system provides both physical actuation and web visualization outputs. Physically, the 2-channel active-LOW relay module toggles high-voltage AC appliance circuits for lamps and fans, while the 16x2 I2C LCD renders real-time entry, exit, and net occupancy counts on-site. Digitally, the web dashboard displays live status cards auto-refreshed every 2 seconds, renders an automated warning banner when appliances remain powered ON in an empty room for 5 seconds or longer, provides interactive manual override buttons (AUTO, ON, OFF), and generates a 2D Highcharts Heatmap chart to analyze weekly occupancy density patterns.

3.4 PROJECT DESIGN

3.4.1 Network Diagram

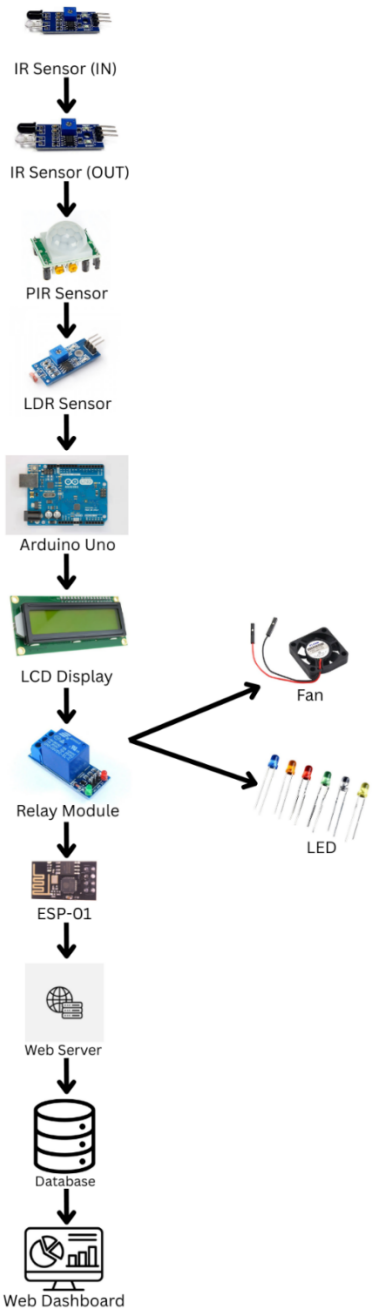


Figure 3.1: Network Design

3.4.2 Use Case Diagram

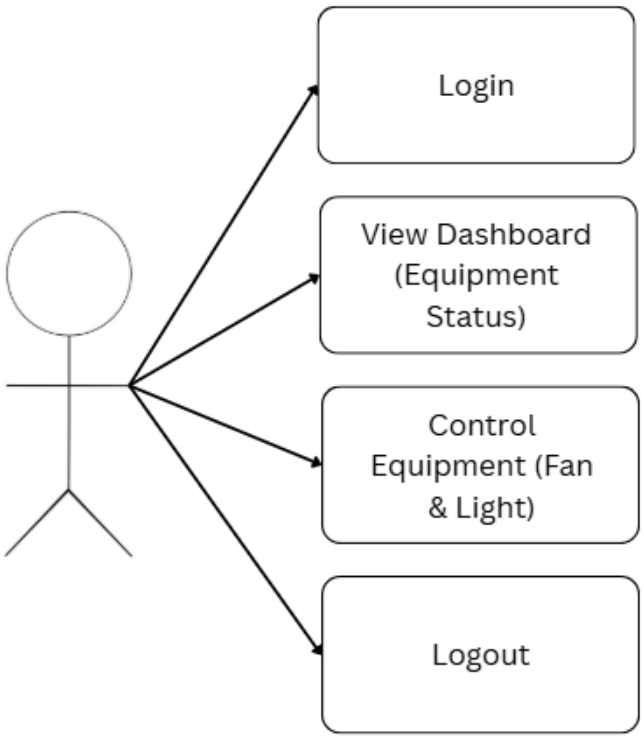


Figure 3.2: Use Case Diagram

3.4.5 ER Diagram

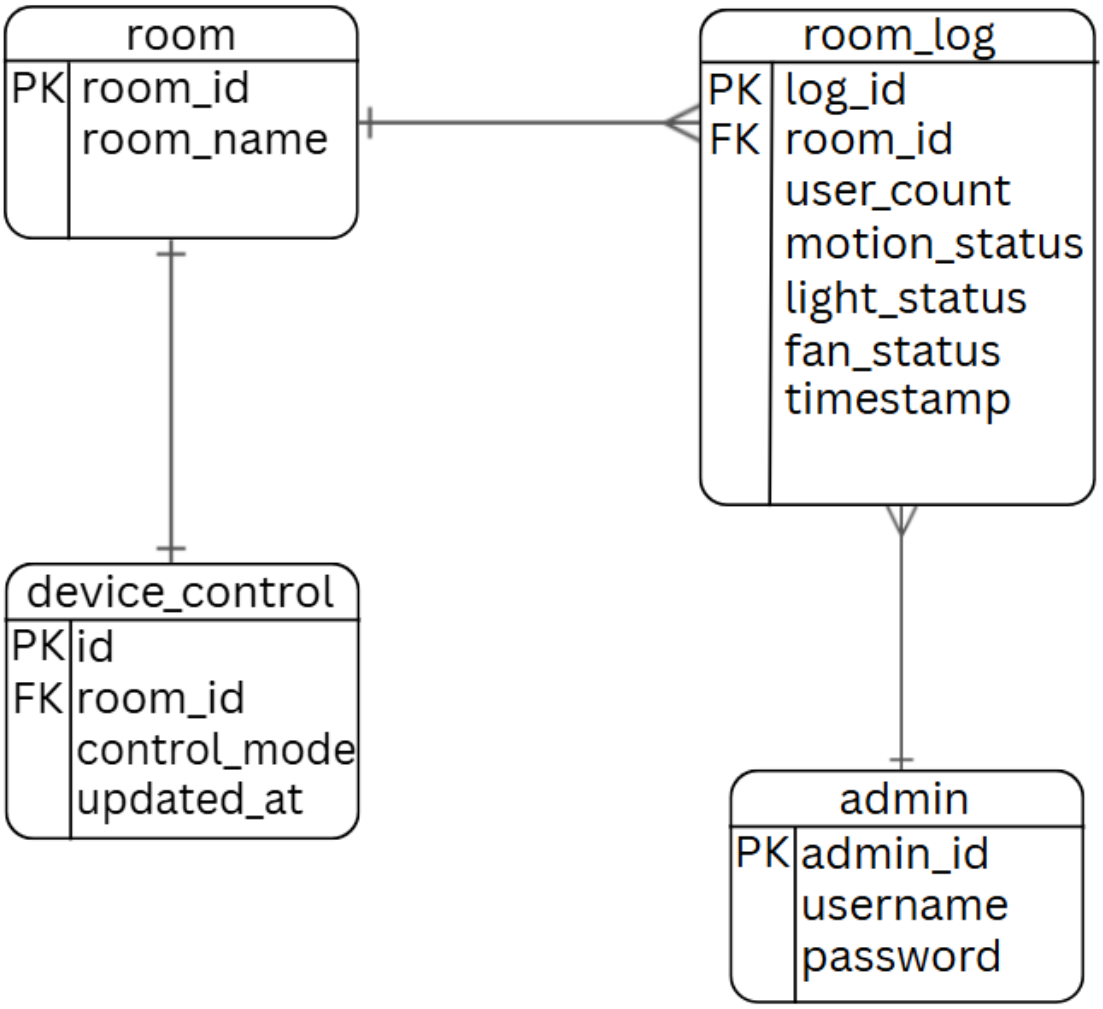


Figure 3.3: ER Diagram

3.4.6 Flowcharts

3.4.6.1 Main IoT System

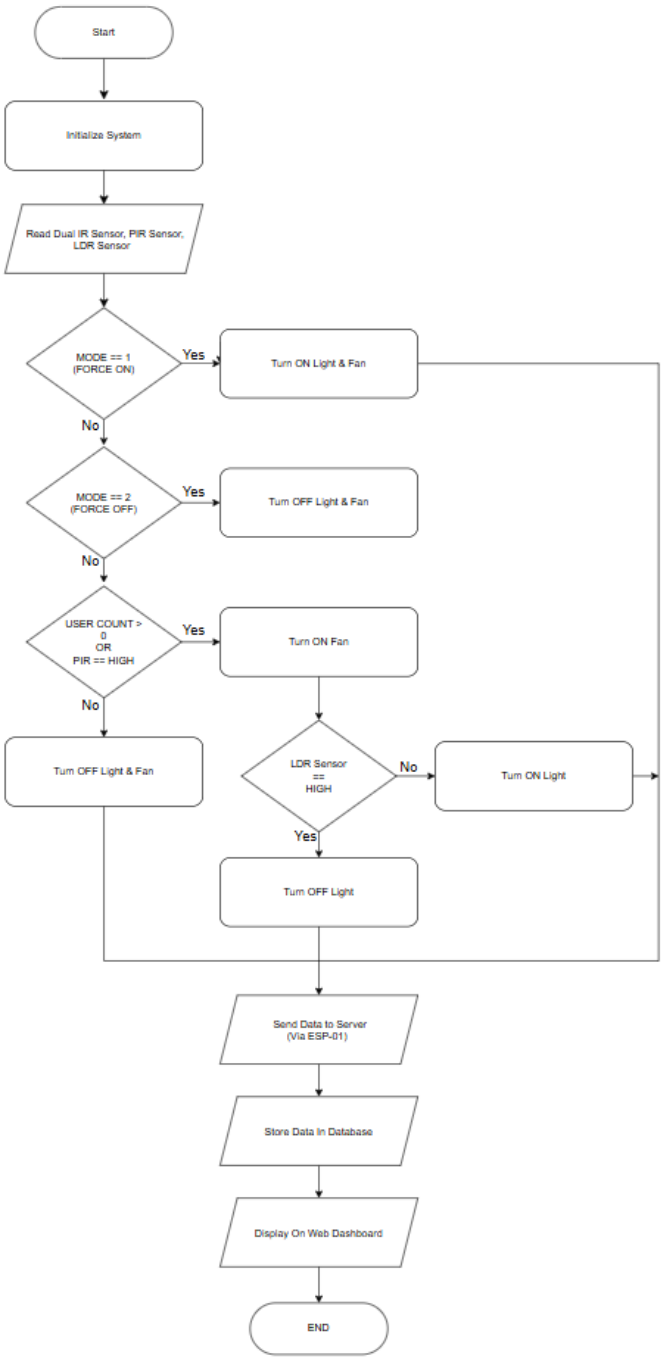


Figure 3.4: Flowchart IoT System

3.4.6.3 Web Dashboard

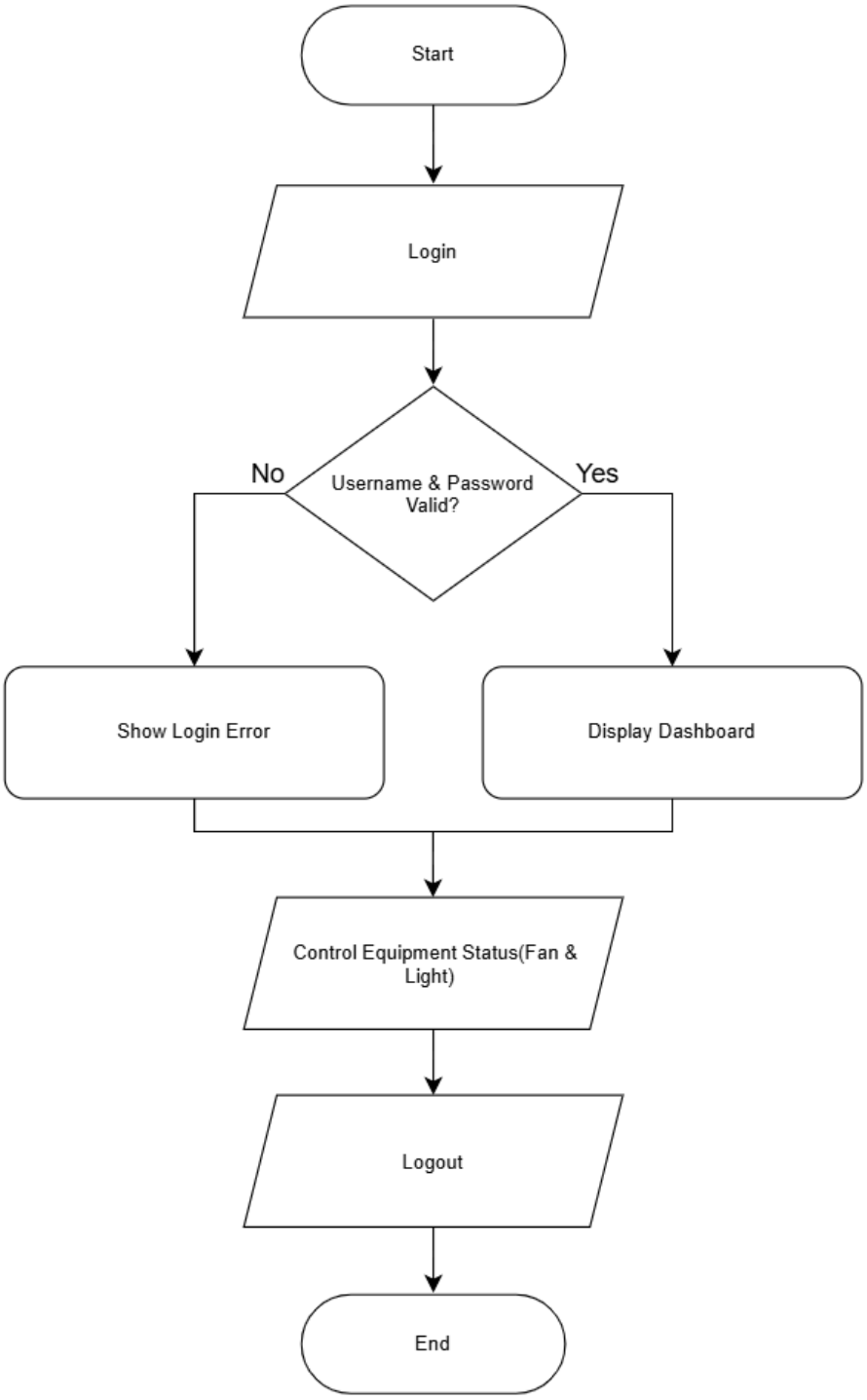


Figure 3.5: Flowchart Admin Login

3.5 GANTT CHART

TASK	MONTHS 2025/2026								
	NOV	DEC	JAN	FEB	MAR	JUN	JULY	AUG	SEP
Title and supervisor agreement									
Requirement Gathering									
Understanding Project Objective									
Understanding Core Features									
Understand Basic Smart Classroom Systems									
Quick Design									
System Planning									
Finding Suitable Hardware									
Finding Suitable Software									
Prototype Development									
Assembling Hardware Components									
Developing Firmware									
Configuring Web Dashboard									
System Testing & Evaluation									
Evaluate Tested Prototype									
Record Feedbacks and Reviews									
Refinement & iteration									
Enhancing the system									
Refining logics									
Optimizing Dashboard									
Completing FYP Presentation									

Figure 3.6: Gant Chart

3.6 CONCLUSION

This chapter presented the methodology and system design of the IoT Smart Classroom project. The System Development Life Cycle (SDLC) was applied to ensure structured development from requirement analysis to implementation and testing.

The conceptual framework and project design explained the integration of IR sensors, Arduino, ESP-01, relay module, LCD I2C display, database, and web dashboard. System diagrams and the ER diagram were developed to illustrate data flow, automation logic, and database structure. The system operates fully automatically without manual dashboard control, in line with the objectives stated in Chapter 1.

This chapter provides the technical foundation for the next chapter, which will discuss system implementation and testing.

CHAPTER 4

PROJECT IMPLEMENTATION

4.1 INTRODUCTION OF CHAPTER

On this chapter shows the results of the IoT Smart Classroom project. It explains what was successfully built and how the system IoT works. The main focus is to show the achievements of the IoT Smart Classroom project based on the objectives set earlier on chapter 1. It will discuss the real output of the IoT system, including dual IR sensor for occupancy counting, Arduino Uno microcontroller as the brain of the project IoT for automated control logic, communication between ESP01 and the Web server to the database for transmit data, web dashboard for admin to display status of the classroom such as light status, fan status, user count and timestamps. This chapter also looks at how wel the system performs and if it solves the problems mentioned in chapter 1.

4.2 PROJECT IMPLEMENTATION

This project was successfully built, the tools that used for this project such as Arduino Uno microcontroller for automated control logic. It will control the other components such as ESP-01 for Wi-Fi connection to transmit the data from Arduino Uno to the Web Server to store on the database and receive control commands from the web. The other components such as dual IR sensor to detect motion from the objects and counting the objects, after IR sensors detect motion from the objects, it will automatically tells the Relay module to turn on the mini fan and LED light because the total objects/users in the classroom is greater than 0 and it will display how many users enter/exit the classroom on the LCD I2C 16x2 display. PIR motion sensor is used to detect human motion inside the classroom, where if no motion is detected for 10 seconds, it

triggers the device timeout, and if no motion is detected for 15 seconds, it automatically resets the count to 0. LDR sensor is used to detect ambient light level with a 1-second debounce delay to prevent light flickering, ensuring the light only turns ON in AUTO mode when the room is dark. The implementation process for this project is divided into three main execution phases: (1) Hardware Assembly and System Setup, (2) Software and Database Development, and (3) Hardware-Software Integration. For the software development, the software that used to write the code for an Arduino Uno Microcontroller is Arduino IDE. The programming language that used to write the code in Arduino IDE is C++, here is all the components for an IoT Smart Classroom communicated between them. Starting from IR Sensors as the input component to an Arduino Uno for this project, this project used dual IR Sensor to detect motion for an object that pass through the dual IR Sensor for entry/exit. The object need to pass through to the dual IR Sensor that are installed on single door for an entry/exit to active the light and fan. If the object past through to only one sensor, the light and fan will not active. There is time for an object to pass through the dual IR Sensor between 5 seconds to active the light and fan. Next Relay module as the output component from an Arduino Uno, whenever the object pass through to the dual IR Sensor that are installed on single door for an entry/exit, it will activates the Relay module to turn ON/OFF the light and fan. LCD I2C 16x2 used to display information about the total users that are in the class and how many users that are already enter or exit from the classroom. Fourthly, ESP-01 Wi-Fi for data transmission from an Arduino Uno to the Web Server to store data on the Database. Arduino Uno Board will sends the data formatted as string containing count, LDR status, light relay status, fan relay status, and PIR status to the ESP-01 via SoftwareSerial. ESP-01 will received the data from Arduino Uno board to sends the data to the Web Server. ESP-01 need the connection of Wi-Fi network before sends it to an HTTP POST request to the web server by carrying the data to the backend PHP script simpan.php. Besides that, ESP-01 will periodically fetch manual override controls from getLedStatus.php every 1.5 seconds and send control commands back to Arduino Uno. ESP-01 also sends periodic data every 60 seconds even if there is no motion. The backend PHP script will receives the data and automatically inserts new record to the table on the MySQL database of IoT Smart Classroom. For the MySQL database for this project, Laragon was used to manage and create the database and tables. The name given for database for this project is smart_classroom, this database contains tables such as admin, room, room_log, and

device_control. The first table are admin to store information such as ID for the admin, username and the password. Secondly, room table to store ID room and room name. Thirdly, room_log table to store important information such as log ID, room ID, user count, motion status, light status, fan status, LDR status and timestamps. Lastly, device_control table stores the control mode status for automated or manual override controls (0 = AUTO, 1 = FORCE ON, 2 = FORCE OFF). room_log and device_control tables will used to display the information to the web dashboard for an admin to view and manage the status of the classroom. When the ESP-01 Wi-Fi sends the occupancy data using HTTP POST request, the backend PHP script called as the simpan.php will receives it on the web server. It process the data and automatically inserts it into the MySQL database table called room_log. The web dashboard was created for admin to monitor the status of the classroom in real time. The file called index.php are the file for the web dashboard that has an automated meta-refresh tag that can reloads the page automatically for every 2 seconds to update the latest data from the database. The web dashboard uses a Bootstrap 5 template to show status for the classroom such as Total Users, LDR Status, Light Status, Fan Status, Motion Status, Last Updated Timestamp, and interactive control buttons (AUTO, ON, OFF) that update device_control through update_control.php. An idle alert banner will automatically appear if user count is 0 but light/fan remains ON for more than 5 seconds. An interactive Highcharts heatmap is also displayed on the dashboard to analyze hourly and daily classroom occupancy density. Before admin can access the web dashboard, they need to make for an user authentication first by register or login. The passwords are encrypted using PHP hashing functions and SQL queries are secured using MySQLi statements to prevent for an unauthorized access and SQL injection vulnerabilities. To ensure all functions work correctly, a testing approach was conducted involving unit testing and system integration testing. Unit testing was performed individually on the dual IR sensors for counting logic, PIR sensor for motion reset, LDR sensor for light sensitivity, and relay module response. System integration testing was carried out to verify end-to-end communication from the Arduino Uno and ESP-01 to the MySQL database via PHP scripts, as well as the responsiveness of manual control triggers and real-time dashboard updates. To successfully implement this IoT project from covering input, processing, output, and communication, all the components must be connected to the Arduino Uno according to a predefined pin mapping architecture:

ESP-01 BOARD	ARDUINO UNO BOARD	ARDUINO UNO BOARD
3V3/VCC	3.3V	-
EN/CH	3.3V	-
GND	GND	-
TX	TX -> 1	-
RX	RX <- 0	-
GPIO1	GND	-
-	GND	RES

Table 4.1: Table of connection ESP-01 to Arduino Uno

Connection pin mapping between Arduino Uno Board and ESP-01 Board to upload the code from Arduino IDE to ESP-01 Board. This pin mapping to enable the ESP-01 into programming mode, make an ESP-01 to upload its own code without depending own Arduino Uno Board. Arduino Uno Board like a bridge to upload the code from Arduino IDE to ESP-01 Board.

Operational Pin Mapping (After Code Upload and Component Integration).

ESP-01	ARDUINO UNO
3V3/VCC	3.3V
EN/CH	3.3V
GND	GND
TX	10
RX	11

Table 4.2: Table of connection ESP-01 to Arduino Uno

BOTH IR SENSOR	ARDUINO UNO
VCC	5V
GND	GND
OUT	2 & 3

Table 4.3: Table of connection Dual IR Sensor to Arduino Uno

LDR SENSOR	ARDUINO UNO
VCC	5V
GND	GND
DO	6

Table 4.4: Table of connection LDR Sensor to Arduino Uno

PIR SENSOR	ARDUINO UNO
VCC	5V
GND	GND
OUT	7

Table 4.5: Table of connection PIR Sensor to Arduino Uno

RELAY MODULE	ARDUINO UNO
VCC	5V
GND	GND
IN1	4
IN2	5

Table 4.6: Table of connection Relay Module to Arduino Uno

LCD I2C 16X2	ARDUINO UNO
VCC	5V
GND	GND
SDA	A4
SCL	A5

Table 4.7: Table of connection LCD I2C 16x2 to Arduino Uno



Figure 4.1: IR, PIR, LDR Sensor

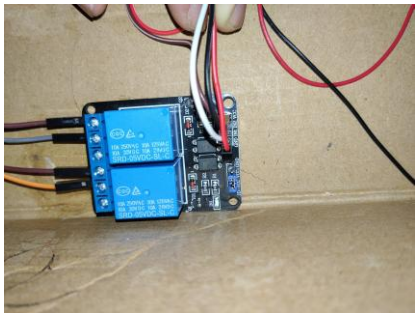


Figure 4.2: Relay Module



Figure 4.3: LCD I2C 16x2

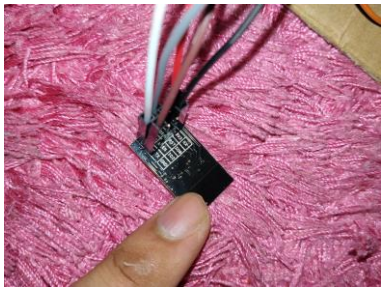


Figure 4.4: ESP-01



Figure 4.5: Fan & Light

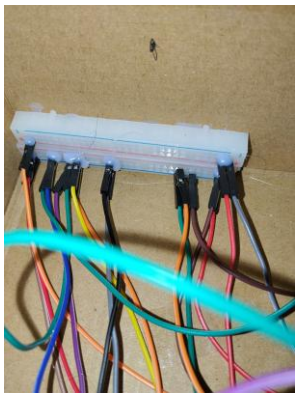


Figure 4.6: Breadboard connector for all components

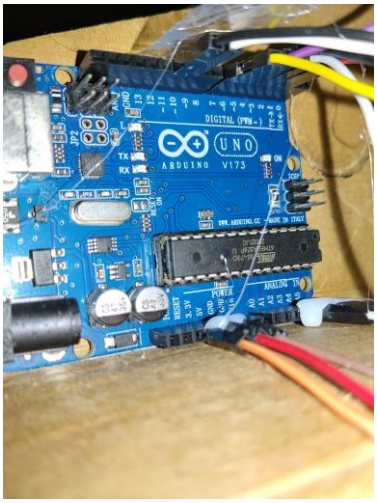


Figure 4.7: Arduino Uno

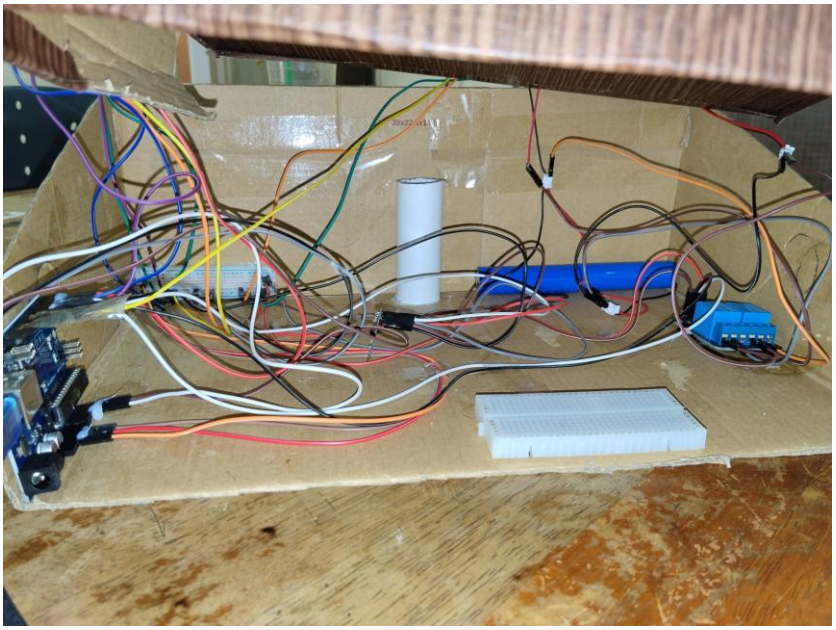


Figure 4.8: Finish setup



Figure 4.9: View from front classroom

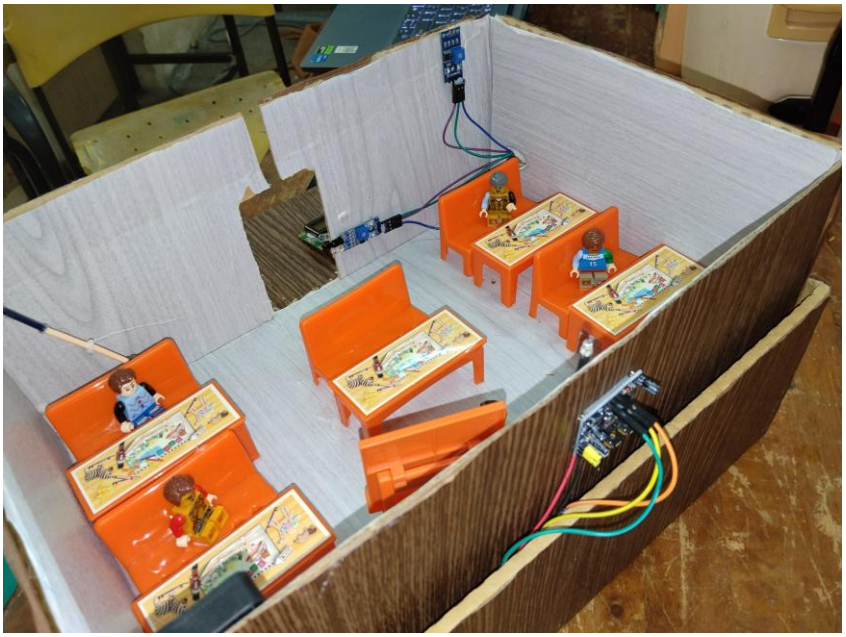


Figure 4.10: View from back classroom

4.3 RESULTS

This section presents the results and findings obtained after the implementation and testing of the IoT Smart Classroom project. The evaluation focuses on assessing hardware accuracy, data transmission reliability, web dashboard responsiveness, and comparing the actual system capabilities with the predefined project objectives outlined in Chapter 1.

4.3.1 Performance Evaluation of Hardware and IoT System

The overall performance of the IoT Smart Classroom system was evaluated across several key hardware components and communication workflows. For the occupancy counting accuracy, tests were conducted by having individuals enter and exit the classroom through the single door setup within the designated 5-second detection window. The dual IR sensors successfully detected direction and updated the LCD display and total user count accurately. Furthermore, the PIR sensor and auto-reset functionality maintained active operation when motion was detected. When the room was empty and no motion was detected for 10 seconds, the device timeout was triggered, and after 15 seconds of continuous no-motion detection, the system successfully auto-reset the user count to zero, correcting any potential counting errors. The LDR sensor accurately detected light and dark conditions, where the 1-second debounce software delay effectively prevented light flickering during transitional ambient light changes, ensuring that lights were triggered in AUTO mode only when the room condition was dark and occupancy or motion was detected. For IoT data communication and relay control, the ESP-01 module successfully transmitted sensor data to the MySQL database via HTTP POST requests to simpan.php while sending background heartbeat data every 60 seconds even without motion changes. Additionally, the ESP-01 fetched manual override states from getLedStatus.php every 1.5 seconds, allowing the relay module to trigger the lamp and fan instantaneously according to AUTO, FORCE ON, or FORCE OFF commands.

4.3.2 Web Dashboard and Analytics

The web dashboard (index.php) successfully provided real-time monitoring and control features for the administrator. Through its real-time auto-refresh mechanism, the page reloaded automatically every 2 seconds to display current user count, LDR status, lamp status, fan status, motion status, and last updated timestamps. The idle device alert system successfully triggered a prominent red alert banner on the dashboard whenever total user count was 0 but appliances such as the lamp or fan remained ON for more than 5 seconds, helping to prevent energy wastage. In addition, the integrated Highcharts heatmap accurately visualized hourly between 00:00 to 23:00 and daily from Sunday to Saturday occupancy trends based on historical logs retrieved from the room_log database table.

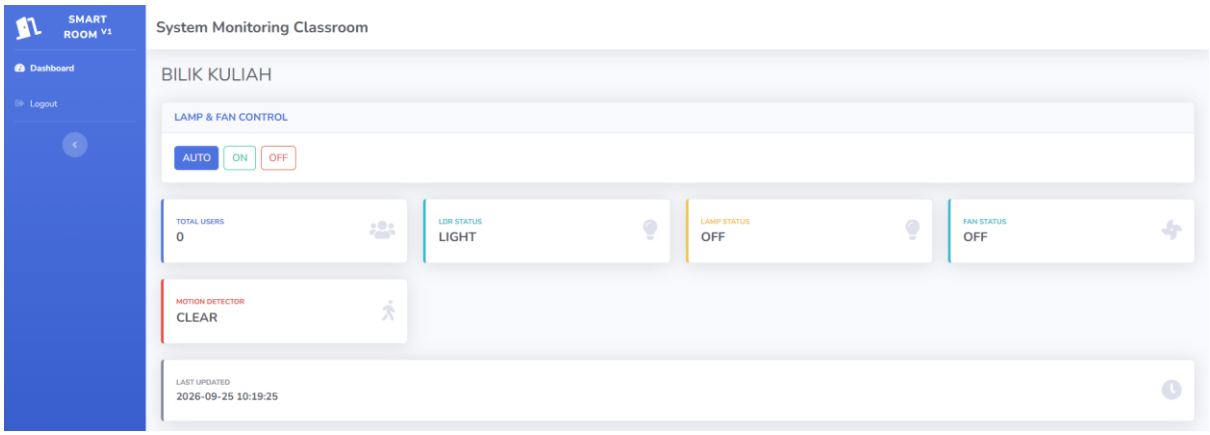


Figure 4.11: Web Dashboard

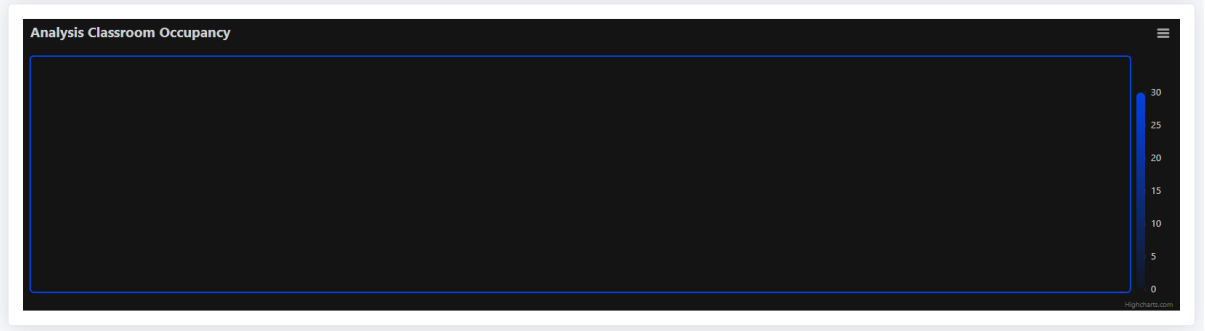


Figure 4.12: Heatmap Chart

4.3.3 Comparison with Project Objectives

The outcomes of the implemented system were compared against the original project objectives set in Chapter 1 to confirm overall project success. For Objective 1, which was to design an automated classroom control system using sensors, the system successfully achieved this by integrating dual IR, PIR, and LDR sensors with the Arduino Uno to automatically control lights and fans based on human presence and ambient light levels. For Objective 2, which aimed to develop Wi-Fi communication and database logging for classroom data, the project successfully achieved it by integrating the ESP-01 module to transmit sensor readings to the Laragon MySQL database room_log table via PHP script simpan.php with a continuous 60-second heartbeat. For Objective 3, which was to construct a web-based dashboard for real-time monitoring, manual overrides, and occupancy analytics, the system successfully achieved it by developing index.php featuring a 2-second auto-refresh rate, interactive manual override buttons for AUTO, FORCE ON, and FORCE OFF modes, idle alert triggers, and a Highcharts classroom density heatmap.

4.3.4 User Testing and System Usability Feedback

To evaluate system usability, a testing session was conducted with administrative users and classroom managers to collect feedback regarding system responsiveness, dashboard interface clarity, and manual control effectiveness. In terms of system responsiveness, data transmission from initial sensor trigger to web dashboard display achieved a fast average latency of under 2 seconds. For manual control override testing, administrative users confirmed that manual trigger commands such as FORCE ON and FORCE OFF successfully overrode automated sensor logic within 1.5 to 2 seconds. Overall user interface clarity feedback showed that test users found the real-time status cards, red idle alert banner, and density heatmap clear and intuitive for daily classroom management.

4.4 CONCLUSION

This section summarizes the key findings from the implementation of the IoT Smart Classroom project, highlighting the success analysis, challenges faced during development, system limitations, and recommendations for future improvements.

4.4.1 Success Analysis

Overall, the project successfully achieved its primary goals by developing a functional IoT-based smart classroom control and monitoring system. The success is demonstrated through the seamless integration of hardware sensors and web interface, where the system accurately performs real-time user counting, automatic lighting and fan control based on room condition and motion, and provides manual web control overrides (AUTO, FORCE ON, FORCE OFF). Furthermore, data transmission via the ESP-01 module to the MySQL database functioned reliably with continuous heartbeat updates. The web dashboard also successfully provided classroom monitoring features, including real-time status updates, idle appliance alert banners, and historical classroom occupancy heatmap analytics for administrative management.

4.4.2 Challenges Faced

During the hardware assembly and system development, several major challenges were encountered regarding component reliability and troubleshooting. Components purchased online, such as the Arduino Uno board, PIR sensor, and LCD display, arrived in imperfect condition and experienced functional issues. The Arduino Uno board suffered from power supply short circuits and intermittent power disconnections whenever touched slightly, while the PIR sensor failed to detect motion consistently and the LCD display failed to light up, requiring replacement units to be bought from a local store. Another major challenge involved the ESP-01 module, which experienced continuous Wi-Fi disconnections and repeated power cycling, making it difficult to determine whether the issue stemmed from a faulty ESP-01 board, loose jumper wire connections, or the microcontroller itself. After thorough troubleshooting, the root cause was identified as the short circuit issue on the initial Arduino Uno board, which unstable power supply to the ESP-01 module and prevented stable Wi-Fi connectivity.

4.4.3 System Limitations

Despite the successful implementation, several hardware and software limitations were identified in the current system. Firstly, the dual IR sensor counting logic cannot accurately register multiple users passing through the doorway simultaneously, as two users walking side-by-side will only be detected as a single entry or exit. Secondly, the PIR motion sensor relies strictly on active movement to keep the light and fan operational; if users inside the classroom remain completely stationary, the system will assume the room is empty and automatically turn off the light and fan. Thirdly, because the ESP-01 module was connected to a mobile phone Wi-Fi hotspot for demonstration, any change in the dynamic hotspot IP address required the hardware wiring to be dismantled to re-enter programming mode on the ESP-01 board. The source code had to be edited with the updated IP address and re-uploaded each time the network configuration changed.

4.4.4 Future Improvements

To address the current limitations and enhance overall functionality, several future improvements are proposed for this project. A major recommended enhancement is the integration of a smart attendance system using barcode scanners to read student identification matrix cards. Instead of merely counting the total number of entries and exits, the system would scan the barcode at the back of student matrix cards prior to entering the classroom. This feature would log specific user identities directly onto the web dashboard, allowing administrators to monitor exact classroom utilization and identify the last recorded users in the event of facility damage or technical issues.

4.5 CONCLUSION OF CHAPTER

In conclusion, this chapter successfully presented the complete project implementation, system performance results, and comprehensive evaluation of the IoT Smart Classroom system. The development process covered the hardware integration of the Arduino Uno, ESP-01 module, dual IR sensors, PIR motion sensor, LDR sensor, relay module, and LCD display, along with the backend PHP server logic, MySQL database management, and the Bootstrap-based admin web dashboard. The performance evaluation confirmed that the system effectively automates user occupancy counting, environmental lighting adjustment, and appliance control, while providing real-time data logging, idle alerts, and classroom density heatmap analytics. Despite encountering hardware assembly challenges and system limitations, appropriate troubleshooting steps and solutions were executed to achieve the project objectives. The successful implementation and key findings of this chapter validate the feasibility of the IoT Smart Classroom system, paving the way for the final chapter, which will summarize the overall project achievements, overall conclusions, and final recommendations for future deployment.

CHAPTER 5

CONCLUSION AND RECOMMENDATION

5.1 INTRODUCTION OF CHAPTER

This chapter summarizes the entire IoT Smart Classroom project, reflecting on the implementation process and highlighting the overall outcomes. It provides a reflection on the lessons learned during system development, outlines practical recommendations for future continuation, and reviews whether the initial project objectives and problem statements defined in Chapter 1 have been successfully addressed. Finally, this chapter concludes with the final summary of the project's contributions to smart classroom management.

5.2 LESSON LEARNED

Throughout the execution of this project, various technical and non-technical skills were acquired during the hardware assembly, troubleshooting, and software development phases. In terms of technical knowledge, important hands-on experience was gained in connecting hardware interfaces between the Arduino Uno microcontroller and individual sensors. Learning how to adjust sensor detection parameters was a key highlight, particularly with the PIR sensor, which features physical potentiometers that can be tuned using a screwdriver to adjust detection range sensitivity and delay time. Additionally, practical knowledge was gained in hardware circuit troubleshooting to pinpoint exact hardware faults, such as identifying power short circuits and disconnections across jumper wires and microcontrollers. In terms of non-technical skills, project management and problem-solving abilities were strengthened. The process taught time management, patience when diagnosing technical errors, real-time error debugging, and systematic documentation methods.

5.3 RECOMMENDATION FOR FUTURE PROJECT CONTINUATION

To improve the system and ensure future project scalability, several practical recommendations are proposed for future developers. Firstly, upgrading the core controller hardware from an Arduino Uno combined with an ESP-01 module to a single ESP32 microcontroller board is highly recommended. The ESP32 already features built-in Wi-Fi connectivity and superior processing power, allowing all component control logic to be written on a single board to create a more compact and reliable circuit without relying on dual-board serial communication. Secondly, integrating a smart attendance feature using student matrix card barcodes would allow the web dashboard to log exact user identities before granting classroom access, enabling administrators to track who is currently using the room and identify responsible users if equipment damage occurs. Thirdly, expanding system scalability from monitoring a single classroom to supporting multi-classroom monitoring on the admin web dashboard would allow administrators to oversee multiple rooms simultaneously. Furthermore, adding an electrical energy loss cost calculator to the web dashboard would visualize potential financial waste caused when appliances are left running unnecessarily. Finally, installing manual physical wall switches inside the classroom alongside the automated sensors would provide a reliable backup option in case of hardware or sensor failures.

5.4 CONCLUSION

In conclusion, the IoT Smart Classroom project successfully met its core objectives by delivering a functional smart control and real-time classroom monitoring system. The final developed prototype addresses the energy wastage and manual monitoring problems identified in Chapter 1 by integrating dual IR sensors, PIR motion sensor, LDR sensor, and relay modules with an Arduino Uno for automated occupancy counting and appliance control. The integration of the ESP-01 module enabled continuous data logging to the MySQL database and real-time dashboard visualization via index.php, complete with manual override controls (AUTO, FORCE ON, FORCE OFF), idle appliance alerts, and Highcharts classroom density heatmaps. The project scope was fully realized, demonstrating significant improvements over manual classroom management.

5.5 CONCLUSION OF CHAPTER

This final chapter closes the report by reinforcing the achievements, technical insights, and overall contributions of the IoT Smart Classroom project. The project demonstrates how IoT technologies, microcontrollers, and web dashboards can be combined to solve real-world energy management and classroom monitoring challenges. It is hoped that this prototype serves as a strong foundation for future smart campus developments and research continuations.

REFERENCES

Pratama, Y., & Hidayat, A. (2021). *Smart classroom automation system*. ResearchGate. https://www.researchgate.net/publication/386373160_Smart_Classroom_Automation_System

Patil, A., & Kulkarni, P. (2018). *Smart classroom system with IoT*. International Journal of Engineering Research & Technology (IJERT). https://www.ijert.org/smart-classroom-system-with-iot

Rahman, I., et al. (2020). *Smart classroom for electricity-saving with integrated IoT system*. ResearchGate. https://www.researchgate.net/publication/356709712_Smart_Classroom_for_Electricity-Saving_with_Integrated_IoT_System

Yusof, N., et al. (2019). *Energy smart saving classroom*. Journal of Education and Learning Studies. https://jurnal.globaleconedu.org/index.php/jels/article/view/212/pdf

Patil, A., & Kulkarni, P. (2018). *Smart classroom system*. International Journal of Engineering Research & Technology. https://www.ijert.org/smart-classroom-system-with-iot

IOP Publishing. (2020). *Smart classroom implementation using IoT*. IOP Conference Series: Materials Science and Engineering. https://iopscience.iop.org/article/10.1088/1757-899X/917/1/012086

ScienceDirect. (2024). *IoT-based smart classroom monitoring system*. Elsevier. https://www.sciencedirect.com/science/article/pii/S2542660524004104

Journal ISI. (2022). *Smart classroom monitoring system*. Journal-ISI. https://journal-isi.org/index.php/isi/article/view/1356

MakersG. (2026). *Sending-sensors-data-from-Arduino-to-MySQL/MakersGroup_sensding_sensors_data_from_arduino_to_mysql_server/MakersGroup_sensding_sensors_data_from_arduino_to_mysql_server.ino at main · MakersG/Sending-sensors-data-from-Arduino-to-MySQL*. GitHub.

https://github.com/MakersG/Sending-sensors-data-from-Arduino-to-MySQL/blob/main/MakersGroup_sensding_sensors_data_from_arduino_to_mysql_server/MakersGroup_sensding_sensors_data_from_arduino_to_mysql_server.ino

ESP8266 NodeMCU HTTP GET and HTTP POST with Arduino IDE | Random Nerd Tutorials. (2020, April 9).

https://randomnerdtutorials.com/esp8266-nodemcu-http-get-post-arduino/

How to Build a Bidirectional Visitor Counter using Arduino. (2025). Circuitdigest.Com. https://circuitdigest.com/microcontroller-projects/how-to-build-a-bidirectional-counter-using-arduino-and-ir-sensors

Santos, R. (2020, May 20). *ESP8266 NodeMCU HTTP POST with Arduino IDE (ThingSpeak and IFTTT.com) | Random Nerd Tutorials*. Random Nerd Tutorials. https://randomnerdtutorials.com/esp8266-nodemcu-http-post-ifttt-thingspeak-arduino/

W3Schools. (2019). *PHP Insert Data Into MySQL*. W3schools.Com. https://www.w3schools.com/php/php_mysql_insert.asp

Bro Code. (2024, May 19). *PHP how to query MySQL database*. YouTube. https://www.youtube.com/watch?v=om_GHl6BrdM

Aktaruzzaman, M. (2024, May 15). *ironman-zaman/login-register*. GitHub. https://github.com/ironman-zaman/login-register

SB Admin 2 - Free Bootstrap Admin Theme. (n.d.). Start Bootstrap. Retrieved October 1, 2026, from https://startbootstrap.com/theme/sb-admin-2

Tech-Trends-Shameer. (2026). *Esp-8266-Projects/Control-LEDs-with-Localhost-MySQL-and-Xampp-Server/control-leds-with-localhost-mysql-and-xampp-server.ino at main · Tech-Trends-Shameer/Esp-8266-Projects*. GitHub. https://github.com/Tech-Trends-Shameer/Esp-8266-Projects/blob/main/Control-LEDs-with-Localhost-MySQL-and-Xampp-Server/control-leds-with-localhost-mysql-and-xampp-server.ino

Heat map Demo | Highcharts.com. (n.d.). Www.Highcharts.Com. Retrieved October 1, 2026, from https://www.highcharts.com/demo/highcharts/heatmap

tfkiot. (2026). *GitHub - tfkiot/esp32-8266_phpMysql_integration_Led_control*. GitHub. https://github.com/tfkiot/esp32-8266_phpMysql_integration_Led_control

APPENDICES

arduinounoboard.ino

```
#include <SoftwareSerial.h>
#include <LiquidCrystal_I2C.h>

// 0x27(Alamat), 16 column, 2 baris
LiquidCrystal_I2C lcd(0x27, 16, 2);

// SoftwareSerial untuk hantar data ke ESP8266 (10 = RX, 11 = TX)
SoftwareSerial espSerial(10, 11);

const int irSensorIn  = 2; // Pin 2 untuk IR sensor (IN)
const int irSensorOut = 3; // Pin 3 untuk IR sensor (OUT)
const int relayLight  = 4; // Pin 4 untuk Relay Lampu
const int relayFan    = 5; // Pin 5 untuk Relay Kipas
const int ldrSensor   = 6; // Pin 6 untuk LDR Sensor
const int pirSensor   = 7; // Pin 7 untuk PIR Sensor

int in_count = 0;
int out_count = 0;
int current_count = 0;
int lastLdrState = -1; // assignkan dengan random value selain 0 dan 1
int lastPirState = -1; // assignkan dengan random value selain 0 dan 1
int lastRelayLightState = -1; // assignkan dengan random value selain 0 dan 1
int lastRelayFanState = -1; // assignkan dengan random value selain 0 dan 1
int webControlStatus = 0; // 0 = AUTO, 1 = FORCE ON, 2 = FORCE OFF
```

```
const unsigned long DEVICE_TIMEOUT = 10000; // 10 Saat untuk matikan Lampu/Kipas jika pir tiada mengesan pergerakan users
const unsigned long RESET_TIMEOUT  = 15000; // 15 Saat untuk reset Count ke 0 jika pir betul2 tiada mengesan pergerakan users
unsigned long lastMotionTime = 0; // untuk pir sensor

// Pembolehubah tambahan khusus untuk hentikan LDR Flickering (Hanya guna standard IF-ELSE)
int stableLdrState = -1;
unsigned long lastLdrDebounceTime = 0;
const unsigned long LDR_DEBOUNCE_DELAY = 1000; // Mesti stabil 1 saat baru tukar status lampu

void setup() {
  Serial.begin(115200);
  espSerial.begin(9600);

  pinMode(irSensorIn, INPUT);
  pinMode(irSensorOut, INPUT);
  pinMode(ldrSensor, INPUT);
  pinMode(pirSensor, INPUT);
  pinMode(relayLight, OUTPUT);
  pinMode(relayFan, OUTPUT);

  // Relay Active LOW: HIGH = OFF, LOW = ON
  digitalWrite(relayLight, HIGH);
  digitalWrite(relayFan, HIGH);

  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print("IN: 0   OUT: 0");
  lcd.setCursor(0, 1);
```

```
  lcd.print("Current: 0");

  lastMotionTime = millis(); // millis() ibarat jam randik
  Serial.println("ARDUINO UNO ACTIVE");
}

void loop() {
  int rawLdrValue = digitalRead(ldrSensor);
  int pirSensorValue = digitalRead(pirSensor);

  // 1. PENAPIS LDR FLICKERING (Debounce LDR supaya tidak bergetar/flicker)
  if (rawLdrValue != lastLdrState) {
    lastLdrDebounceTime = millis();
    lastLdrState = rawLdrValue;
  }

  if ((millis() - lastLdrDebounceTime) > LDR_DEBOUNCE_DELAY) {
    if (rawLdrValue != stableLdrState) {
      stableLdrState = rawLdrValue;
      sendDataToESP(stableLdrState);
    }
  }

  // Jika waktu mula pembacaan, assign stableLdrState nilai semasa
  if (stableLdrState == -1) {
    stableLdrState = rawLdrValue;
  }

  // 2. KEMASKINI PEMASA PIR SENSOR
  if (pirSensorValue == HIGH) { // mula kiraan bagi lastMotionTime apabila pir sensor
terima pergerakan
    lastMotionTime = millis();
  }
```

```
  if (pirSensorValue != lastPirState) { // update value terkini untuk pir sensor
    lastPirState = pirSensorValue;
     applyRelayLogic(stableLdrState, pirSensorValue); // untuk membuat perubahan value serta merta jika ada perubahan value pada kedua sensor kepada relay
    sendDataToESP(stableLdrState); // sentiasa menghantar value terkini kepada esp
  }

  if (millis() - lastMotionTime >= RESET_TIMEOUT) { // jika lastmotiontime lebih daripada resettimeout maka nilai count akan direset kepada 0
    if (current_count > 0) {
      in_count = 0;
      out_count = 0;
      current_count = 0;

      updateDisplay();
      sendDataToESP(stableLdrState);
      Serial.println("The classroom is empty(RESET)");
    }
  }

  // 3. TERIMA ARAHAN KAWALAN DARI ESP8266 (WEB DASHBOARD)
  if (espSerial.available() > 0) {
     String inputData = espSerial.readStringUntil('\n'); // terima input daripada web ke esp
    inputData.trim();
    if (inputData.startsWith("CTRL:")) {
       int newControlStatus = inputData.substring(5).toInt(); // mengambil value index yang ke 5 selepas "CTRL:"

       if (newControlStatus != webControlStatus) { // update status web control yang terkini
        webControlStatus = newControlStatus;
```

```
      applyRelayLogic(stableLdrState, pirSensorValue);
      sendDataToESP(stableLdrState);
    }
   }
  }

  applyRelayLogic(stableLdrState, pirSensorValue); // untuk membuat perubahan value serta merta jika ada perubahan value pada kedua sensor kepada relay

   int currentLightState = digitalRead(relayLight); // baca value terkini untuk relay lampu
  int currentFanState   = digitalRead(relayFan); // baca value terkini untuk relay kipas

  if (currentLightState != lastRelayLightState || currentFanState != lastRelayFanState) { // update value terkini untuk kedua relay
    lastRelayLightState = currentLightState;
    lastRelayFanState   = currentFanState;
    sendDataToESP(stableLdrState);
  }

  int statusIn  = digitalRead(irSensorIn);
  int statusOut = digitalRead(irSensorOut);

  // LOGIK OBJEK MASUK (Sensor IN -> Sensor OUT)
  if (statusIn == LOW && statusOut == HIGH) {
    unsigned long startTime = millis();
    bool isSuccess = false;

    while (millis() - startTime < 5000) {
      if (digitalRead(irSensorOut) == LOW) {
        isSuccess = true;
        break;
      }
```

```
    }

    if (isSuccess) {
      in_count++;
      updateDisplay();
      lastMotionTime = millis();

      Serial.print("Objek MASUK | Jumlah Terkini: ");
      Serial.println(current_count);

      applyRelayLogic(stableLdrState, digitalRead(pirSensor));
      sendDataToESP(stableLdrState);

      unsigned long clearTime = millis();
     while ((digitalRead(irSensorIn) == LOW || digitalRead(irSensorOut) == LOW) &&
(millis() - clearTime < 2000)) {
        delay(10);
      }
    }
  }

  // LOGIK OBJEK KELUAR (Sensor OUT -> Sensor IN)
  else if (statusOut == LOW && statusIn == HIGH) {
    unsigned long startTime = millis();
    bool isSuccess = false;

    while (millis() - startTime < 5000) {
      if (digitalRead(irSensorIn) == LOW) {
        isSuccess = true;
        break;
      }
    }
```

```
    if (isSuccess) {
      if (in_count > out_count) {
        out_count++;
        updateDisplay();
      }

      Serial.print("Objek KELUAR | Jumlah Terkini: ");
      Serial.println(current_count);

      applyRelayLogic(stableLdrState, digitalRead(pirSensor));
      sendDataToESP(stableLdrState);

      unsigned long clearTime = millis();
      while ((digitalRead(irSensorIn) == LOW || digitalRead(irSensorOut) == LOW) &&
(millis() - clearTime < 2000)) {
        delay(10);
      }
    }
  }
}

// Fungsi Kemaskini Relay mengikut Logik Baharu
void applyRelayLogic(int ldrSensorValue, int pirSensorValue) {
  if (webControlStatus == 1) {
    // FORCE ON dari Web
    digitalWrite(relayLight, LOW);
    digitalWrite(relayFan, LOW);
  }
  else if (webControlStatus == 2) {
    // FORCE OFF dari Web
    digitalWrite(relayLight, HIGH);
    digitalWrite(relayFan, HIGH);
  }
```

```
  else {
    // MOD AUTO:
    bool isDeviceTimeout = (millis() - lastMotionTime >= DEVICE_TIMEOUT);

     // Memastikan jika current_count > 0, alat tidak dimatikan sesuka hati oleh DEVICE_TIMEOUT
    bool activeCondition = false;
    if (current_count > 0) {
      activeCondition = true;
    } else if (pirSensorValue == HIGH && !isDeviceTimeout) {
      activeCondition = true;
    }

    if (activeCondition) {
      digitalWrite(relayFan, LOW); // Kipas ON

      if (ldrSensorValue == HIGH) {
        digitalWrite(relayLight, LOW);  // Bilik Gelap -> Lampu ON
      } else {
        digitalWrite(relayLight, HIGH); // Bilik Terang -> Lampu OFF
      }

    } else {
      // Jika count == 0 DAN tiada pergerakan melepasi 10 saat -> MATI
      digitalWrite(relayLight, HIGH);
      digitalWrite(relayFan, HIGH);
    }
  }
}

// Fungsi Hantar Data ke ESP8266 dengan Standard IF-ELSE
void sendDataToESP(int ldrSensorValue) {
  int relayLightStatus;
```

```
  if (digitalRead(relayLight) == LOW) {
    relayLightStatus = 1;
  } else {
    relayLightStatus = 0;
  }

  int relayFanStatus;
  if (digitalRead(relayFan) == LOW) {
    relayFanStatus = 1;
  } else {
    relayFanStatus = 0;
  }

  int pirStatus;
  if (digitalRead(pirSensor) == HIGH) {
    pirStatus = 1;
  } else {
    pirStatus = 0;
  }

  // Format: COUNT,LDR,RELAY_LIGHT,RELAY_FAN,PIR
  espSerial.print(current_count);
  espSerial.print(",");
  espSerial.print(ldrSensorValue);
  espSerial.print(",");
  espSerial.print(relayLightStatus);
  espSerial.print(",");
  espSerial.print(relayFanStatus);
  espSerial.print(",");
  espSerial.println(pirStatus);
}

// Fungsi Kemaskini Paparan LCD
```

```
void updateDisplay() {
  current_count = in_count - out_count;

  lcd.setCursor(4, 0);
  lcd.print("   ");
  lcd.setCursor(4, 0);
  lcd.print(in_count);

  lcd.setCursor(13, 0);
  lcd.print("   ");
  lcd.setCursor(13, 0);
  lcd.print(out_count);

  lcd.setCursor(9, 1);
  lcd.print("   ");
  lcd.setCursor(9, 1);
  lcd.print(current_count);
}
```

esp01board.ino

```
#include <ESP8266WiFi.h> // library untuk akses wifi
#include <ESP8266HTTPClient.h> // untuk post data (count) ke database/web dashboard

#define WIFI_SSID "hi" // nama wifi/ssid
#define WIFI_PASSWORD "afiq12345" // password bagi wifi/ssid
#define SERVER_URL "http://10.93.119.30/smart_classroom/test/simpan.php" // esp8266 hantar data kepada url/server berikut melalui kaedah http post
#define GET_LED_URL "http://10.93.119.30/smart_classroom/test/getLedStatus.php" // url untuk dapatkan status kawalan manual dari web
```

```
String postData;

unsigned long lastFetchTime = 0; // pembolehubah untuk kawal selang masa semak status
unsigned long lastLogTime = 0;   // pembolehubah untuk kawal hantaran log berkala

// Pembolehubah untuk simpan nilai bacaan terkini
int receivedCount = 0;
int ldrVal = 0;
int relayLightVal = 0; // Pembolehubah untuk simpan status Relay Lampu
int relayFanVal = 0;   // Pembolehubah untuk simpan status Relay Kipas
int pirVal = 0;        // Pembolehubah untuk simpan status PIR Sensor

void checkWiFiConnection()
{
  while (WiFi.status() != WL_CONNECTED) // periksa status wifi(ssid, pass, 2.4ghz), jika tepat maka connect
  {
    Serial.println("Connecting to Wi-Fi...");
    delay(1000);
  }
}

// Fungsi tambahan untuk semak arahan manual dari getLedStatus.php
void checkServerControl()
{
  if (millis() - lastFetchTime > 1500) // semak status dari web setiap 1.5 saat
  {
    lastFetchTime = millis();
    WiFiClient client;
    HTTPClient http;

    http.begin(client, GET_LED_URL);
```

```
    int httpCode = http.GET();

    if (httpCode == HTTP_CODE_OK)
    {
      String response = http.getString();
      response.trim();

      // Hantar arahan status ke Arduino Uno melalui Serial
      Serial.print("CTRL:");
      Serial.println(response);
    }
    http.end();
  }
}

void setup()
{
  Serial.begin(9600); // aktifkan serial monitor pada bauderate yang rendah
  Serial.println("Communication Started\n");

  WiFi.mode(WIFI_STA); // jadikan esp8266 sebagai client spt laptop/pc/smartphone untuk connect
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD); // mulakan connection

  checkWiFiConnection();

  Serial.println("Connected to Wi-Fi");
  Serial.print("IP Address: ");
  Serial.println(WiFi.localIP()); // memaparkan nilai LAN Ip address bagi wifi tersebut
}

void loop()
{
```

```
  checkWiFiConnection(); // Pastikan Wi-Fi sentiasa terhubung sebelum buat apa-apa

  checkServerControl(); // Semak status kawalan manual dari web server

  if (Serial.available() > 0) // Hanya baca jika ada data masuk dari Arduino Uno
  {
   String inputData = Serial.readStringUntil('\n'); // menerima data daripada arduino
   inputData.trim();

   // Abaikan jika data yang diterima ialah mesej kawalan yang dipantulkan
   if (inputData.length() > 0 && !inputData.startsWith("CTRL:"))
   {
            // Asingkan nilai jika format dipisah dengan koma: "COUNT,LDR,RELAY_LIGHT,RELAY_FAN,PIR"
     int firstComma = inputData.indexOf(',');
     int secondComma = inputData.indexOf(',', firstComma + 1);
     int thirdComma = inputData.indexOf(',', secondComma + 1);
     int fourthComma = inputData.indexOf(',', thirdComma + 1);

     if (firstComma > 0 && secondComma > 0 && thirdComma > 0) {
      receivedCount = inputData.substring(0, firstComma).toInt();
      ldrVal = inputData.substring(firstComma + 1, secondComma).toInt();
      relayLightVal = inputData.substring(secondComma + 1, thirdComma).toInt();

      if (fourthComma > 0) {
       relayFanVal = inputData.substring(thirdComma + 1, fourthComma).toInt();
       pirVal = inputData.substring(fourthComma + 1).toInt();
      } else {
       relayFanVal = inputData.substring(thirdComma + 1).toInt();
      }
     } else {
      if (firstComma > 0) {
       // Jika Arduino Uno hanya hantar 2 nilai (COUNT, LDR)
```

```
    receivedCount = inputData.substring(0, firstComma).toInt();
    ldrVal = inputData.substring(firstComma + 1).toInt();
  } else {
    receivedCount = inputData.toInt();
  }
}

Serial.print("OBJECT DETECTED, COUNT = ");
Serial.println(receivedCount);

// start hantar data
WiFiClient client;
HTTPClient http;

// hantar data drpd arduino uno sebagai POST request
postData = "room_id=1&count=" + String(receivedCount) +
           "&ldr=" + String(ldrVal) +
           "&relaylight=" + String(relayLightVal) +
           "&relayfan=" + String(relayFanVal) +
           "&pir=" + String(pirVal);

// papran di serial monitor
Serial.print("Sending data: ");
Serial.println(postData);

// proses hantar data
http.begin(client, SERVER_URL);
http.addHeader("Content-Type", "application/x-www-form-urlencoded");

// hantar data drpd arduino uno sebagai POST request
int httpCode = http.POST(postData); // 200/404/403/500/-1

if (httpCode > 0)
```

```
    {
     Serial.print("HTTP Response code: ");
     Serial.println(httpCode);

     if (httpCode == HTTP_CODE_OK) // 200 == 200
     {
      String response = http.getString();
      Serial.println("Server response: " + response);
     }
    }
    else
    {
     Serial.print("HTTP POST request failed with error code: ");
     Serial.println(httpCode);

      if (httpCode == HTTPC_ERROR_CONNECTION_REFUSED) // -1 server
tutup/ip salah/ tak connect wifi
     {
      Serial.println("Connection refused by the server.");
     }
     else
     {
      if (httpCode == HTTP_CODE_NOT_FOUND) // 404 berjaya connect dengan
server tetapi file tidak jumpa
      {
       Serial.println("Server resource not found.");
      }
      else
      {
       Serial.println("Unknown error occurred."); // error yang lain selain 2 di atas
      }
     }
    }
```

```
    http.end();
    lastLogTime = millis(); // reset timer berkala bila ada hantaran dari sensor
   }
  }

   // --- HANTARAN BERKALA (SETIAP 60 SAAT) WALAU TIADA PERGERAKAN ---
  if (millis() - lastLogTime > 60000)
  {
   lastLogTime = millis();

   WiFiClient client;
   HTTPClient http;

   postData = "room_id=1&count=" + String(receivedCount) +
              "&ldr=" + String(ldrVal) +
              "&relaylight=" + String(relayLightVal) +
              "&relayfan=" + String(relayFanVal) +
              "&pir=" + String(pirVal);

   http.begin(client, SERVER_URL);
   http.addHeader("Content-Type", "application/x-www-form-urlencoded");
   http.POST(postData);
   http.end();
  }
}
```

index.php

```
<?php
// Set zon masa Malaysia di peringkat teratas skrip
date_default_timezone_set('Asia/Kuala_Lumpur');

include "header.template.php";

$servername = "localhost";
$username   = "root";
$password   = "";
$dbname     = "smart_classroom";

$conn = mysqli_connect($servername, $username, $password, $dbname);

if (!$conn) {
   die("Connection failed: " . mysqli_connect_error());
}

// 1. Fetch record dari database
$sql = "SELECT * FROM room_log WHERE room_id = 1 ORDER BY log_id DESC
LIMIT 1";
$result = mysqli_query($conn, $sql);

$count = 0;
$lightStatus = 0;
$fanStatus = 0;
$lightStatusText = "OFF";
$fanStatusText = "OFF";
$motionStatusText = "CLEAR";
$timestamp = "-";
$ldrStatusText = "LIGHT";
```

```
if ($result) {
    if (mysqli_num_rows($result) > 0) {
        $row = mysqli_fetch_assoc($result);
        $count = (int)$row['user_count'];
        $lightStatus = (int)$row['light_status'];
        $fanStatus = (int)$row['fan_status'];

        if (isset($row['timestamp'])) {
            $timestamp = $row['timestamp'];
        } else {
            $timestamp = "-";
        }

        if ($lightStatus == 1) {
            $lightStatusText = "ON";
        } else {
            $lightStatusText = "OFF";
        }

        if ($fanStatus == 1) {
            $fanStatusText = "ON";
        } else {
            $fanStatusText = "OFF";
        }

        if ($row['ldr_status'] == 0) {
            $ldrStatusText = "LIGHT";
        } else {
            $ldrStatusText = "DARK";
        }

        if ($row['motion_status'] == 1) {
            $motionStatusText = "DETECTED";
```

```
        } else {
            $motionStatusText = "CLEAR";
        }
    }
}

// 2. Semak Mod Kawalan
$controlMode = 0;
$sql_ctrl = "SELECT control_mode FROM device_control WHERE room_id = 1 LIMIT 1";
$res_ctrl = mysqli_query($conn, $sql_ctrl);

if ($res_ctrl) {
    if (mysqli_num_rows($res_ctrl) > 0) {
        $row_c = mysqli_fetch_assoc($res_ctrl);
        $controlMode = (int)$row_c['control_mode'];
    }
}

// 3. LOGIK PEMERIKSAAN PERANTI TERBIAR (> 5 SAAT)
$showAlert = false;

if ($count == 0) {
    if ($lightStatus == 1 || $fanStatus == 1) {
        if ($timestamp != "-") {
            $logTime = strtotime($timestamp);
            $currentTime = time();

            // Gunakan abs() untuk elak nilai negatif akibat perbezaan jam server
            $timeDifference = abs($currentTime - $logTime);

            // Jika tiada orang & lampu/kipas ON melebihi 5 saat
            if ($timeDifference >= 5) {
```

```
            $showAlert = true;
        }
    }
}

// 4. Fetch data Heatmap (Maksimum Minggu Semasa)
$heatmapDataArray = array();
$sql_heatmap = "SELECT
                (DAYOFWEEK(timestamp) - 1) AS day_index,
                HOUR(timestamp) AS hour_index,
                MAX(user_count) AS max_users
            FROM room_log
            WHERE room_id = 1
             AND YEARWEEK(timestamp, 1) = YEARWEEK(CURDATE(), 1)
            GROUP BY day_index, hour_index";

$res_heatmap = mysqli_query($conn, $sql_heatmap);

if ($res_heatmap) {
    if (mysqli_num_rows($res_heatmap) > 0) {
        while ($h_row = mysqli_fetch_assoc($res_heatmap)) {
            $h_day  = (int)$h_row['day_index'];
            $h_hour = (int)$h_row['hour_index'];
            $h_max  = (int)$h_row['max_users'];
            $heatmapDataArray[] = array($h_hour, $h_day, $h_max);
        }
    }
}

$jsonHeatmapData = json_encode($heatmapDataArray);

mysqli_close($conn);
```

```
$autoBtnClass = "btn-outline-primary";
if ($controlMode == 0) {
   $autoBtnClass = "btn-primary";
}

$onBtnClass = "btn-outline-success";
if ($controlMode == 1) {
   $onBtnClass = "btn-success";
}

$offBtnClass = "btn-outline-danger";
if ($controlMode == 2) {
   $offBtnClass = "btn-danger";
}
?>

<!DOCTYPE html>
<html>
<head>
   <!-- Refresh halaman setiap 2 saat untuk mengemas kini pengiraan masa secara langsung -->
   <meta http-equiv="refresh" content="2">
   <title>Room Status</title>
   <script src="https://code.highcharts.com/highcharts.js"></script>
   <script src="https://code.highcharts.com/modules/heatmap.js"></script>
   <script src="https://code.highcharts.com/modules/exporting.js"></script>
   <script src="https://code.highcharts.com/modules/accessibility.js"></script>
</head>
<body>

   <div class="container-fluid">
      <h2 class="h3 mb-4 text-gray-800">BILIK KULIAH</h2>
```

```
<!-- PAPARAN BANNER AMARAN (BILA TERBIAR > 5 SAAT) -->
<?php if ($showAlert) { ?>
    <div class="alert alert-danger alert-dismissible fade show shadow mb-4" role="alert">
        <h4 class="alert-heading"><i class="fas fa-exclamation-triangle"></i> AMARAN: PERANTI TERBIAR!</h4>
        <p class="mb-0">Tiada pengguna dikesan di dalam kelas tetapi lampu/kipas masih menyala melebihi 5 saat. Sila tekan butang <strong>OFF</strong> di bawah untuk mematikan perkakas.</p>
    </div>
<?php } ?>

<!-- Panel Kawalan Manual Override -->
<div class="row mb-4">
  <div class="col-12">
    <div class="card shadow">
      <div class="card-header font-weight-bold text-primary">LAMP & FAN CONTROL</div>
      <div class="card-body d-flex gap-2">
        <form action="update_control.php" method="POST" class="mr-2" style="display:inline;">
          <input type="hidden" name="mode" value="0">
          <button type="submit" class="btn <?php echo $autoBtnClass; ?>">AUTO</button>
        </form>
        <form action="update_control.php" method="POST" class="mr-2" style="display:inline;">
          <input type="hidden" name="mode" value="1">
          <button type="submit" class="btn <?php echo $onBtnClass; ?>">ON</button>
        </form>
```

```
                                <form action="update_control.php" method="POST"
style="display:inline;">
                    <input type="hidden" name="mode" value="2">
                        <button type="submit" class="btn <?php echo $offBtnClass;
?>">OFF</button>
                </form>
              </div>
            </div>
          </div>

      <div class="row">

        <!-- Total Users Card -->
        <div class="col-xl-3 col-md-6 mb-4">
          <div class="card border-left-primary shadow h-100 py-2">
            <div class="card-body">
              <div class="row no-gutters align-items-center">
                <div class="col mr-2">
                  <div class="text-xs font-weight-bold text-primary text-uppercase
mb-1">Total Users</div>
                    <div class="h5 mb-0 font-weight-bold text-gray-800"><?php echo
$count; ?></div>
                </div>
                <div class="col-auto">
                  <i class="fas fa-users fa-2x text-gray-300"></i>
                </div>
              </div>
            </div>
          </div>

        <!-- LDR Status Card -->
```

```
        <div class="col-xl-3 col-md-6 mb-4">
          <div class="card border-left-info shadow h-100 py-2">
            <div class="card-body">
              <div class="row no-gutters align-items-center">
                <div class="col mr-2">
                  <div class="text-xs font-weight-bold text-info text-uppercase mb-
1">LDR Status</div>
                  <div class="h5 mb-0 font-weight-bold text-gray-800"><?php echo
$ldrStatusText; ?></div>
                </div>
                <div class="col-auto">
                  <i class="fas fa-lightbulb fa-2x text-gray-300"></i>
                </div>
              </div>
            </div>
          </div>
        </div>

        <!-- Light Status Card -->
        <div class="col-xl-3 col-md-6 mb-4">
          <div class="card border-left-warning shadow h-100 py-2">
            <div class="card-body">
              <div class="row no-gutters align-items-center">
                <div class="col mr-2">
                  <div class="text-xs font-weight-bold text-warning text-uppercase
mb-1">Lamp Status</div>
                  <div class="h5 mb-0 font-weight-bold text-gray-800"><?php echo
$lightStatusText; ?></div>
                </div>
                <div class="col-auto">
                  <i class="fas fa-lightbulb fa-2x text-gray-300"></i>
                </div>
              </div>
```

```
                </div>
            </div>
        </div>

        <!-- Fan Status Card -->
        <div class="col-xl-3 col-md-6 mb-4">
            <div class="card border-left-info shadow h-100 py-2">
                <div class="card-body">
                    <div class="row no-gutters align-items-center">
                        <div class="col mr-2">
                            <div class="text-xs font-weight-bold text-info text-uppercase mb-1">Fan Status</div>
                            <div class="h5 mb-0 font-weight-bold text-gray-800"><?php echo $fanStatusText; ?></div>
                        </div>
                        <div class="col-auto">
                            <i class="fas fa-fan fa-2x text-gray-300"></i>
                        </div>
                    </div>
                </div>
            </div>
        </div>

        <!-- Motion Detector Card -->
        <div class="col-xl-3 col-md-6 mb-4">
            <div class="card border-left-danger shadow h-100 py-2">
                <div class="card-body">
                    <div class="row no-gutters align-items-center">
                        <div class="col mr-2">
                            <div class="text-xs font-weight-bold text-danger text-uppercase mb-1">Motion Detector</div>
                            <div class="h5 mb-0 font-weight-bold text-gray-800"><?php echo $motionStatusText; ?></div>
```

```
                    </div>
                    <div class="col-auto">
                        <i class="fas fa-walking fa-2x text-gray-300"></i>
                    </div>
                </div>
            </div>
        </div>

        <!-- Last Updated Card -->
        <div class="col-xl-12 col-md-12 mb-4">
            <div class="card border-left-secondary shadow h-100 py-2">
                <div class="card-body">
                    <div class="row no-gutters align-items-center">
                        <div class="col mr-2">
                            <div class="text-xs font-weight-bold text-secondary text-uppercase
mb-1">Last Updated</div>
                            <div class="h6 mb-0 font-weight-bold text-gray-800"><?php echo
$timestamp; ?></div>
                        </div>
                        <div class="col-auto">
                            <i class="fas fa-clock fa-2x text-gray-300"></i>
                        </div>
                    </div>
                </div>
            </div>
        </div>

        <!-- HEATMAP CHART CARD -->
        <div class="col-xl-12 col-md-12 mb-4">
            <div class="card shadow">
                <div class="card-body">
                    <div id="container" style="height: 400px;"></div>
```

```
                </div>
            </div>
        </div>

    </div>
</div>

<!-- SCRIPT HIGHCHARTS HEATMAP -->
<script>
Highcharts.Templating.helpers.substr = function(s, from, length) {
    return String(s).substr(from, length);
};

var dbHeatmapData = <?php echo $jsonHeatmapData; ?>;

Highcharts.chart('container', {
    chart: {
        type: 'heatmap',
        plotBorderColor: '#0443E1',
        plotBorderWidth: 2,
        plotBorderRadius: 5,
        marginBottom: 25
    },

    title: {
        text: 'Analysis Classroom Occupancy',
        align: 'left'
    },

    xAxis: {
        categories: [
            '00:00', '01:00', '02:00', '03:00', '04:00', '05:00',
            '06:00', '07:00', '08:00', '09:00', '10:00', '11:00',
```

```
            '12:00', '13:00', '14:00', '15:00', '16:00', '17:00',
            '18:00', '19:00', '20:00', '21:00', '22:00', '23:00'
        ],
        opposite: true,
        labels: {
            autoRotation: [0, -90],
            style: { fontSize: '0.65em' }
        },
        lineWidth: 1,
        lineColor: '#0443E1'
    },

    yAxis: {
        categories: ['Ahad', 'Isnin', 'Selasa', 'Rabu', 'Khamis', 'Jumaat', 'Sabtu'],
        title: null,
        reversed: true,
        labels: {
            rotation: 0,
            style: { fontSize: '0.65em' }
        }
    },

    accessibility: {
        point: {
                    descriptionFormat: 'Pada hari {series.yAxis.categories.(y)} jam
{series.xAxis.categories.(x)}, bilangan maksimum ialah {value} pengguna.'
        }
    },

    colorAxis: {
        min: 0,
        max: 30,
        startOnTick: false,
```

```
        endOnTick: false,
        tickPixelInterval: 50,
        minColor: '#0443E111',
        maxColor: '#0443E1',
        tickColor: '#ffffff',
        tickLength: 0.1,
        tickWidth: 6,
        gridLineWidth: 0,
        marker: {
            symbol: 'circle',
            color: 'transparent',
            lineColor: '#fff',
            lineWidth: 3
        }
    },

    legend: {
        align: 'right',
        layout: 'vertical',
        symbolRadius: 6,
        margin: 0,
        verticalAlign: 'bottom',
        symbolHeight: 270
    },

    tooltip: {
        useHTML: true,
        format: `<div style="border-left: 3px solid {point.color}; padding-left: 8px;">
                Hari: <b>{series.yAxis.categories.(point.y)}</b><br>
                Masa: <b>{series.xAxis.categories.(point.x)}</b><br>
                Maksimum Pelajar: <b>{point.value} orang</b></div>`,
        padding: 12
    },
```

```
    series: [{
        name: 'Kepadatan Maksimum',
        borderColor: '#0443E1',
        borderWidth: 1,
        data: dbHeatmapData,
        dataLabels: {
            enabled: true,
            style: { fontSize: '0.85em' }
        }
    }],

    responsive: {
        rules: [{
            condition: { maxWidth: 500 },
            chartOptions: {
                yAxis: {
                    labels: {
                        rotation: 0,
                        format: '{substr value 0 3}'
                    }
                }
            }
        }]
    }
});
</script>

</body>
</html>

<?php
include "footer.template.php";
```

```
?>
```

simpan.php

```
<?php
$servername = "localhost";
$username   = "root";
$password   = "";
$dbname     = "smart_classroom";

$conn = mysqli_connect($servername, $username, $password, $dbname);

if (!$conn) {
   die("Connection failed: " . mysqli_connect_error());
}

if (isset($_POST['count'])) {

   if (isset($_POST['room_id'])) {
      $roomId = (int)$_POST['room_id'];
   } else {
      $roomId = 1;
   }

   $count = (int)$_POST['count'];

   if (isset($_POST['ldr'])) {
      $ldr = (int)$_POST['ldr'];
   } else {
      $ldr = 0;
   }

   if (isset($_POST['relaylight'])) {
      $lightStatus = (int)$_POST['relaylight'];
   } else {
      $lightStatus = 0;
   }

   if (isset($_POST['relayfan'])) {
      $fanStatus = (int)$_POST['relayfan'];
   } else {
      $fanStatus = 0;
   }

   // Mengambil nilai PIR terus jika dihantar oleh ESP8266
   if (isset($_POST['pir'])) {
      $motionStatus = (int)$_POST['pir'];
   } else {
```

```
        if ($count > 0) {
            $motionStatus = 1;
        } else {
            $motionStatus = 0;
        }
    }

    $sql = "INSERT INTO room_log (room_id, user_count, motion_status,
light_status, fan_status, ldr_status)
            VALUES ($roomId, $count, $motionStatus, $lightStatus, $fanStatus, $ldr)";

    if (mysqli_query($conn, $sql)) {
        echo "Data saved successfully";
    } else {
        echo "Error: " . mysqli_error($conn);
    }

} else {
    echo "Insufficient information";
}

mysqli_close($conn);
?>
```

getLedStatus.php

```
<?php
$servername = "localhost";
$username   = "root";
$password   = "";
$dbname     = "smart_classroom";

$conn = mysqli_connect($servername, $username, $password, $dbname);

if (!$conn) {
    die("0");
}

$roomId = 1;
$sql    = "SELECT control_mode FROM device_control WHERE room_id =
$roomId LIMIT 1";
$result = mysqli_query($conn, $sql);

if ($result && mysqli_num_rows($result) > 0) {
    $row = mysqli_fetch_assoc($result);
    echo $row['control_mode'];
} else {
    echo "0";
}
```

```
mysqli_close($conn);
?>
```

update_control.php

```
<?php
$servername = "localhost";
$username   = "root";
$password   = "";
$dbname     = "smart_classroom";

$conn = mysqli_connect($servername, $username, $password, $dbname);

if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}

if (isset($_POST['mode'])) {
    $mode = (int)$_POST['mode']; // 0 = AUTO, 1 = FORCE ON, 2 = FORCE OFF
    $roomId = 1; // Default Room ID

    // Semak samada rekod room_id sudah wujud
    $checkSql = "SELECT * FROM device_control WHERE room_id = $roomId";
    $result   = mysqli_query($conn, $checkSql);

    if (mysqli_num_rows($result) > 0) {
        // Kemaskini mod sedia ada
        $sql = "UPDATE device_control SET control_mode = $mode WHERE room_id
= $roomId";
    } else {
        // Masukkan rekod baru jika tiada
        $sql = "INSERT INTO device_control (room_id, control_mode) VALUES
($roomId, $mode)";
    }

    mysqli_query($conn, $sql);
}

mysqli_close($conn);

// Hantar pengguna kembali ke Dashboard (index.php)
header("Location: index.php");
exit();
?>
```

login.php

```
<?php
session_start();
// Redirect to index page if already logged in
if (isset($_SESSION["admin"])) {
    header("Location: index.php");
    exit();
}
?>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Admin Login</title>
    <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@5.2.2/dist/css/bootstrap.min.css"
integrity="sha384-
Zenh87qX5JnK2Jl0vWa8Ck2rdkQ2Bzep5IDxbcnCeuOxjzrPF/et3URy9Bv1WTRi"
crossorigin="anonymous">
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="container mt-5" style="max-width: 500px;">
        <?php
        if (isset($_POST["login"])) {
            $username = trim($_POST["username"]);
            $password = $_POST["password"];

            require_once "database.php";

            // Using prepared statement for security against SQL Injection
            $sql = "SELECT * FROM admin WHERE username = ?";
            $stmt = mysqli_prepare($conn, $sql);
            mysqli_stmt_bind_param($stmt, "s", $username);
            mysqli_stmt_execute($stmt);
            $result = mysqli_stmt_get_result($stmt);
            $user = mysqli_fetch_array($result, MYSQLI_ASSOC);

            if ($user) {
                // Verify hashed password
                if (password_verify($password, $user["password"])) {
                    $_SESSION["admin"] = $user["username"];
                    $_SESSION["admin_id"] = $user["admin_id"];
                    header("Location: index.php");
                    exit();
                } else {
```

```
                echo "<div class='alert alert-danger'>Password does not match!</div>";
            }
        } else {
            echo "<div class='alert alert-danger'>Username not found!</div>";
        }
    }
    ?>
    <form action="login.php" method="post">
      <div class="form-group mb-3">
        <input type="text" placeholder="Enter Username" name="username"
class="form-control" required>
      </div>
      <div class="form-group mb-3">
        <input type="password" placeholder="Enter Password" name="password"
class="form-control" required>
      </div>
      <div class="form-btn mb-3">
        <input type="submit" value="Login" name="login" class="btn btn-primary
w-100">
      </div>
    </form>
    <div><p>Not registered yet? <a href="registration.php">Register
Here</a></p></div>
  </div>
</body>
</html>
```

logout.php

```
<?php
session_start();
session_destroy();
header("Location: login.php");
?>
```

registration.php

```
<?php
session_start();
if (isset($_SESSION["admin"])) {
   header("Location: index.php");
   exit();
}
?>
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
```

```
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Admin Registration</title>
    <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@5.2.2/dist/css/bootstrap.min.css"
integrity="sha384-
Zenh87qX5JnK2Jl0vWa8Ck2rdkQ2Bzep5IDxbcnCeuOxjzrPF/et3URy9Bv1WTRi"
crossorigin="anonymous">
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="container mt-5" style="max-width: 500px;">
        <?php
        if (isset($_POST["submit"])) {
            $username = trim($_POST["username"]);
            $password = $_POST["password"];
            $passwordRepeat = $_POST["repeat_password"];

            $passwordHash = password_hash($password, PASSWORD_DEFAULT);

            $errors = array();

            // Input Validation
            if (empty($username) OR empty($password) OR empty($passwordRepeat)) {
                array_push($errors, "All fields are required.");
            }
            if (strlen($username) > 50) {
                array_push($errors, "Username cannot exceed 50 characters.");
            }
            if (strlen($password) < 8) {
                array_push($errors, "Password must be at least 8 characters long.");
            }
            if ($password !== $passwordRepeat) {
                array_push($errors, "Passwords do not match.");
            }

            require_once "database.php";

            // Check if username already exists using prepared statements
            $sql = "SELECT * FROM admin WHERE username = ?";
            $stmt = mysqli_stmt_init($conn);
            if (mysqli_stmt_prepare($stmt, $sql)) {
                mysqli_stmt_bind_param($stmt, "s", $username);
                mysqli_stmt_execute($stmt);
                $result = mysqli_stmt_get_result($stmt);
                if (mysqli_num_rows($result) > 0) {
                    array_push($errors, "Username already exists!");
                }
            }
```

```
        // Display errors if any
        if (count($errors) > 0) {
            foreach ($errors as $error) {
                echo "<div class='alert alert-danger'>$error</div>";
            }
        } else {
            // Insert new admin record
            $sql = "INSERT INTO admin (username, password) VALUES (?, ?)";
            $stmt = mysqli_stmt_init($conn);
            $prepareStmt = mysqli_stmt_prepare($stmt, $sql);
            if ($prepareStmt) {
                mysqli_stmt_bind_param($stmt, "ss", $username, $passwordHash);
                mysqli_stmt_execute($stmt);
                echo "<div class='alert alert-success'>Admin registered successfully! <a
href='login.php'>Login Here</a></div>";
            } else {
                die("Something went wrong on the server.");
            }
        }
    }
    ?>
    <form action="registration.php" method="post">
        <div class="form-group mb-3">
            <input type="text" class="form-control" name="username"
placeholder="Username:" required>
        </div>
        <div class="form-group mb-3">
            <input type="password" class="form-control" name="password"
placeholder="Password:" required>
        </div>
        <div class="form-group mb-3">
            <input type="password" class="form-control" name="repeat_password"
placeholder="Repeat Password:" required>
        </div>
        <div class="form-btn mb-3">
            <input type="submit" class="btn btn-primary w-100" value="Register"
name="submit">
        </div>
    </form>
    <div>
        <p>Already Registered? <a href="login.php">Login Here</a></p>
    </div>
  </div>
</body>
</html>
```

header.template.php

```
<!DOCTYPE html>
<html lang="ms">

<head>

   <meta charset="utf-8">
   <meta http-equiv="X-UA-Compatible" content="IE=edge">
   <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-
fit=no">
   <title>Smart Classroom - Dashboard</title>

   <!-- Custom fonts for this template-->
   <link href="vendor/fontawesome-free/css/all.min.css" rel="stylesheet"
type="text/css">
   <link
href="https://fonts.googleapis.com/css?family=Nunito:200,200i,300,300i,400,400i,60
0,600i,700,700i,800,800i,900,900i" rel="stylesheet">

   <!-- Custom styles for this template-->
   <link href="css/sb-admin-2.min.css" rel="stylesheet">

</head>

<body id="page-top">

   <!-- Page Wrapper -->
   <div id="wrapper">

      <!-- Sidebar -->
      <ul class="navbar-nav bg-gradient-primary sidebar sidebar-dark accordion"
id="accordionSidebar">

         <!-- Sidebar - Brand -->
         <a class="sidebar-brand d-flex align-items-center justify-content-center"
href="index.php">
            <div class="sidebar-brand-icon rotate-n-15">
               <i class="fas fa-door-open"></i>
            </div>
            <div class="sidebar-brand-text mx-3">Smart Room <sup>v1</sup></div>
         </a>

         <!-- Divider -->
         <hr class="sidebar-divider my-0">

         <!-- Nav Item - Dashboard -->
         <li class="nav-item active">
            <a class="nav-link" href="index.php">
```

```
                <i class="fas fa-fw fa-tachometer-alt"></i>
                <span>Dashboard</span></a>
            </li>

            <!-- Tambah Pautan Logout di sini -->
            <li class="nav-item">
                <a class="nav-link" href="logout.php">
                    <i class="fas fa-fw fa-sign-out-alt"></i>
                    <span>Logout</span></a>
            </li>

            <!-- Divider -->
            <hr class="sidebar-divider d-none d-md-block">

            <!-- Sidebar Toggler (Sidebar) -->
            <div class="text-center d-none d-md-inline">
                <button class="rounded-circle border-0" id="sidebarToggle"></button>
            </div>

        </ul>
        <!-- End of Sidebar -->

        <!-- Content Wrapper -->
        <div id="content-wrapper" class="d-flex flex-column">

            <!-- Main Content -->
            <div id="content">

                <!-- Topbar -->
                <nav class="navbar navbar-expand navbar-light bg-white topbar mb-4
static-top shadow">

                    <!-- Sidebar Toggle (Topbar) -->
                    <button id="sidebarToggleTop" class="btn btn-link d-md-none rounded-
circle mr-3">
                        <i class="fa fa-bars"></i>
                    </button>

                    <!-- Topbar Title -->
                    <h1 class="h4 mb-0 text-gray-800 font-weight-bold">System
Monitoring Classroom</h1>

                </nav>
                <!-- End of Topbar -->
```

footer.template.php

```
        <!-- Footer -->
        <footer class="sticky-footer bg-white">
          <div class="container my-auto">
            <div class="copyright text-center my-auto">
              <span>IoT Smart Classroom 2026</span>
            </div>
          </div>
        </footer>
        <!-- End of Footer -->

      </div>
      <!-- End of Content Wrapper -->

    </div>
    <!-- End of Page Wrapper -->

    <!-- Scroll to Top Button-->
    <a class="scroll-to-top rounded" href="#page-top">
      <i class="fas fa-angle-up"></i>
    </a>

    <!-- Logout Modal-->
    <div class="modal fade" id="logoutModal" tabindex="-1" role="dialog" aria-
labelledby="exampleModalLabel"
      aria-hidden="true">
      <div class="modal-dialog" role="document">
        <div class="modal-content">
          <div class="modal-header">
            <h5 class="modal-title" id="exampleModalLabel">Ready to
Leave?</h5>
            <button class="close" type="button" data-dismiss="modal" aria-
label="Close">
              <span aria-hidden="true">×</span>
            </button>
          </div>
          <div class="modal-body">Select "Logout" below if you are ready to end
your current session.</div>
          <div class="modal-footer">
            <button class="btn btn-secondary" type="button" data-
dismiss="modal">Cancel</button>
            <a class="btn btn-primary" href="login.html">Logout</a>
          </div>
        </div>
      </div>
    </div>
```

```
    <!-- Bootstrap core JavaScript-->
    <script src="vendor/jquery/jquery.min.js"></script>
    <script src="vendor/bootstrap/js/bootstrap.bundle.min.js"></script>

    <!-- Core plugin JavaScript-->
    <script src="vendor/jquery-easing/jquery.easing.min.js"></script>

    <!-- Custom scripts for all pages-->
    <script src="js/sb-admin-2.min.js"></script>

    <!-- Page level plugins -->
    <script src="vendor/chart.js/Chart.min.js"></script>

    <!-- Page level custom scripts -->
    <script src="js/demo/chart-area-demo.js"></script>
    <script src="js/demo/chart-pie-demo.js"></script>

</body>

</html>
```

database.php

```
<?php

$hostName = "localhost";
$dbUser = "root";
$dbPassword = "";
$dbName = "smart_classroom"; // Replace this with your actual database name in
phpMyAdmin

$conn = mysqli_connect($hostName, $dbUser, $dbPassword, $dbName);

if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}

?>
```

style.css

```
body{
    padding:50px;
}
.container{
    max-width: 600px;
    margin:0 auto;
    padding:50px;
    box-shadow: rgba(100, 100, 111, 0.2) 0px 7px 29px 0px;
```

```
}
.form-group{
    margin-bottom:30px;
}
```